

论文题目 私有网盘应用系统的设计与实现

专业学位类别 工 程 硕 士

学 号 201651060134

作 者 姓 名 付 波

指 导 教 师 杨国武 教授

分类号 _____ 密级 _____

UDC ^{注 1} _____

学 位 论 文

私有网盘应用系统的设计与实现

(题名和副题名)

付 波

(作者姓名)

指导教师 杨国武 教 授

电子科技大学 成 都

(姓名、职称、单位名称)

申请学位级别 硕士 专业学位类别 工 程 硕 士

工程领域名称 计算机技术

提交论文日期 2018 年 10 月 答辩日期 2018 年 11 月 29 日

学位授予单位和日期 电子科技大学 2018 年 12 月

答辩委员会主席 _____

评阅人 _____

注 1：注明《国际十进分类法 UDC》的类号。

Design and Implementation of Private Network Disk Application System

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline: **Master of Engineering**

Author: **Fu Bo**

Supervisor: **Prof. Guo wuyang**

School: **School of Computer Science & Engineering**

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名： 付波

日期：2018年12月4日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定，有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（保密的学位论文在解密后应遵守此规定）

作者签名： 付波

导师签名： 杨国武

日期：2018年12月4日

摘 要

随着互联网的发展,近年来大数据技术和云存储技术在全球学术界、工业界和国家政府中备受关注。越来越多的国外研究单位和研究人员正在投入云存储和大数据的研究和开发。目前,许多国内网络磁盘(包括个人网络磁盘和企业网络磁盘)主要提供公共云服务,只有少数网络磁盘供应商提供私有云服务。

可见云环境下的大数据存储不管是对企业还是个人用户变得尤为重要。然而分散的存储方式往往意味着资料面临着设备损坏、病毒和网络攻击以及人为的删除和破坏。数据海量的快速增长让云存储系统面临的巨大压力和挑战。为了解决企业进行日常的业务管理,根据各种业务应用系统的特性很方便地综合调整资源的分配,解决基础设施资源的复用、通用业务能力开放、业务承载等一系列问题。尤其是网盘数据存储安全性和存储问题。本论文从以下几个工作来开展研究设计并实现完整的私有网盘系统:

(1) 从系统业务需求出发,在客户现场调研分析用户真实的业务需求,基于持续与客户业务人员详细沟通交流的过程,阶段性的获取到了系统完整的需求文档,包括功能需求和非功能性需求。并进行的大量定制化的功能服务。实现用户在系统的操作使用过程中拥有简单、统一的管理平台,易于系统调动和分配资源的目标。

(2) 采用 JAVA 技术为基础,开发工具是 eclipse,运行于 JVM 环境中。在需求分析的基础上分别利用 SpringCloud 技术和 PowerDesigner 对系统进行系统架构设计和数据库设计。Web 容器采用 WebShpere 支持分布式部署,Rides 缓存框架,Hadoop 存储等平台技术支撑完成了私有网盘应用系统的功能开发和设计。

(3) 利用单元模块测试和系统集成测试的方法来对系统整体进行测试,验证系统的主要功能模块是否正常实现。

关键词: 私有云, 网盘, Spring, Java

ABSTRACT

With the development of the Internet, big data technology and cloud storage technology have been attracted much attention in the global academia, industry and national government in recent years. More and more foreign research institutes and researchers are engaged in the research and development of cloud storage and big data. At present, many domestic network disks (including personal network disks and enterprise network disks) mainly provide public cloud services, but only a few network disks providers provide private cloud services.

It can be seen that big data storage in cloud environment is particularly important either for enterprises or individual users. However, decentralized storage often means that data is facing equipment damage, virus and network attacks, and artificial deletion and destruction. With the rapid growth of data, the cloud storage system is facing tremendous pressure and challenges. In order to solve the daily business management of enterprises, it is very convenient to adjust the allocation of resources comprehensively according to the characteristics of various business application systems, and solve a series of problems, such as reuse of infrastructure resources, opening of general business capabilities, and business carrying, etc. Especially, network disk data storage security and storage problems. In this thesis, the following researches are carried out to design and implement a complete private network disk system:

(1) It is started from the business needs of the system. The real business needs of users are investigated and analyzed in the customer field. Based on the continuous communication with customer business personnel, documents required by the system are obtained in stages, including functional requirements and non-functional requirements. And a big number of customized functional services are conducted. The goals that users have a simple and unified management platform in the process of

operating the system are achieved.

(2) Based on JAVA technology, the development tool is eclipse, which runs in JVM environment. On the basis of requirement analysis, Spring Cloud technology and PowerDesigner are used to design the system architecture and database. WebShpere is used as a web container to support distributed deployment. Rides caching framework and Hadoop storage platform technology are used to support the development and design of private network disk application system.

(3) Unit test, module test and system integration test are used to test the whole system. It aims to verify whether the main functional modules of the system are properly implemented.

Keywords: Private cloud, network disk, Spring, Java

目 录

第一章 绪论	1
1.1 课题背景及意义.....	1
1.1.1 课题来源与依据.....	1
1.1.2 课题研究的价值与意义.....	1
1.2 国内外研究现状.....	2
1.3 课题研究的主要内容.....	2
1.3.1 课题研究目标.....	2
1.3.2 课题研究的主要内容.....	3
1.4 论文结构.....	4
第二章 系统研发的技术原理	5
2.1 JAVAEE 系统设计开发.....	5
2.1.1 面向对象编程.....	5
2.1.2 面向对象的概念与特征.....	6
2.1.3 面向对象的分析.....	7
2.1.4 面向对象的设计.....	7
2.1.5 面向对象的编程.....	8
2.2 基于 UML 的软件建模.....	8
2.3 JAVAEE 开发技术.....	10
2.3.1 JDK.....	10
2.3.2 JavaEE.....	12
2.4 本章小结.....	12
第三章 需求分析	13
3.1 功能需求分析.....	13
3.1.1 统一存储服务.....	13
3.1.2 文件视图浏览.....	14
3.1.3 文档在线预览.....	15
3.1.4 图片预览播放.....	16
3.1.5 音视频动画播放.....	17
3.1.6 在线解压缩.....	21

3.1.7 文件上传下载.....	22
3.2 非功能需求分析.....	23
3.2.1 支持复杂文件预览.....	23
3.2.2 精准高品质文件转换.....	23
3.2.3 支持超大文件转换.....	24
3.2.4 极速分页加载.....	24
3.2.5 无限极快速解压缩.....	24
3.2.6 安全性、可靠性、可扩展性.....	24
3.3 本章小结.....	25
第四章 系统架构设计与实现.....	26
4.1 系统架构.....	26
4.2 系统详细设计.....	27
4.2.1 系统组件设计.....	27
4.2.2 系统流程设计.....	32
4.2.3 系统接口设计.....	38
4.2.4 数据库设计.....	43
4.3 系统编码实现.....	47
4.3.1 多文档上传下载组件实现.....	47
4.3.2 文档转换引擎实现.....	48
4.3.3 文档在线阅读器实现.....	50
4.3.4 流媒体服务器实现.....	51
4.3.5 音视频转码引擎实现.....	53
4.3.6 图片播放器实现.....	57
4.3.7 音视频播放器实现.....	58
4.4 系统集成环境.....	60
4.5 本章小结.....	62
第五章 系统测试.....	63
5.1 系统测试.....	63
5.1.1 测试目标.....	63
5.1.2 测试环境.....	64
5.1.3 功能测试.....	66
5.1.4 性能测试.....	69
5.2 测试分析.....	69

第六章 总结与展望.....	71
6.1 全文总结.....	71
6.2 后续工作展望.....	72
致谢.....	73
参考文献.....	74

第一章 绪论

1.1 课题背景及意义

1.1.1 课题来源与依据

计算机技术是现代经济发展的核心动力，也是发展最快的领域之一。我国已经进入信息化时代。随着计算机技术的不断发展，无论是在生活中还是企事业单位中计算机技术扮演者举足轻重的作用。互联网企业飞速发展、传统行业的规模逐步的扩大，数据大小已从 PB 级增加到 EB 级，而且它仍在不断增长。这就意味着对大数据的存储需求越来越迫切，在这样的环境下企业会使用深入的数据分析来支持其利润的增长。由此可见一个功能强大的企业私有云存储系统能够有效避免云存储系统本身在文件、数据海量的快速增长上面临的巨大压力和挑战。当今云环境下的大数据存储对企业 and 用户变得尤为重要。

由于大数据处理和云存储需求的紧迫性和重要性，近年来大数据技术和云存储技术在全球学术界，工业界和国家政府中备受关注。本课题即是在此基础上，以私有网盘系统在现实生活中的需求为例，探索研究如何构建并开发一个实现文档，图片，视频在线预览甚至是一些程序源代码文件，都可以实现免下载的在线预览和存储功能软件系统。

1.1.2 课题研究的价值与意义

私有网盘应用系统研发的初心主要是解决传统平台下表单、公文、流程、电子邮件等业务模块的附件管理在线预览，以及为其他所有子系统提供附件的在线预览服务。本系统可以预览的附件种类很多，包括微软 Office 整套办公系列文档、金山 WPS 整套办公系列文档，开源 OpenOffice 整套办公系列文档；ZIP 或 RAR 压缩包，图片文件，绝大多数音频视频 Flash 动画文件；甚至是一些程序源码文件，都可以实现存储、下载以及在线预览、在线解压缩功能。

本课题以企业私有网盘平台为例进行需求分析和业务开展，目前课题的研究已得到该企业的支持并为后期软件的实施提供了必要条件，课题的研究成果能够得到及时的应用，收到第一手的研究反馈资料，对于课题的进一步研究有很大的促进作用。

1.2 国内外研究现状

自云计算发展以来,已经衍生出两种形式:公有云和私有云。目前,许多国内网络磁盘(包括个人网络磁盘和企业网络磁盘)主要提供公共云服务,只有少数网络磁盘供应商提供私有云服务。云环境中的大数据存储属于云计算和大数据基础支撑设计,开发空间巨大。作为基础架构,迫切需要云环境中的大数据存储。越来越多的国外研究单位和研究人员正在投入云存储和大数据的研究和开发^[1]。前些年,大数据安全存储系统产品的成熟度很差,甚至几乎没有相关产品的出现。但是近两年在云存储软件产品行业得到相应发展。目前之所以发展缓慢尚未大规模普及,主要是需要解决好海量数据及大文件在云端环境存储的主要问题。企业在运营中保存了大量的商业机密信息数据,此时数据存储安全在企业中扮演着十分重要的角色,倘若数据存储不当或安全受到威胁这就会给云环境下企业知识资产带来严重的侵害和影响。

云计算服务自 SaaS 出现以来,受到整个 IT 行业的关注,云计算也进入了一个飞速发展的时代。随后,很多知名的互联网公司开始从不同角度提供不同级别的云计算服务。云服务不仅仅局限于互联网的市场范围。还能将传统的自建 IT 模式转变为使用公共云服务,云服务将真正进入其行业的成熟期。

公共云服务通常包括 IaaS, PaaS 和 SaaS 服务。IaaS 是一种云平台的服务,是移动互联网和信息化未来的发展方向。移动互联网和其他云计算的发展和应用将导致对 IaaS 的行业需求的持续发展,到那时大数据存储和云计算也将推动 IaaS 的快速演进。IaaS 也是一种硬件集群服务模式,需要在基础设施和长期运营经验方面花费大量资源成本。PaaS 服务更像移动互联网未来的“操作系统”,在目前云计算和应用业务中处于最活跃的领域。与 IaaS 服务的区别, PaaS 服务会给应用程序和开发人员带来更强的业务支撑,所以 PaaS 服务的侧重不在于直观的经济层面。更加注重建立和开拓一个紧密的工业生态系统。 SaaS 服务是最成熟的云服务类型。传统的计算机软件行业厂商将软件副本作为主要销售业务模型, SaaS 服务运用 Web 体系和 SOA 分层架构。采用 Internet 互联网提供多租户和定制化的应用功能来满足用户各种业务场景,大范围缩短了软件销售行业的渠道链条。将行业软件开发商从软件产品研发生产厂商升级为应用服务运营维护厂商。

1.3 课题研究的主要内容

1.3.1 课题研究目标

本课题是基于企业专用网盘系统的需求为基础。通过软件系统功能和结构设

计、数据库设计来实现主要功能模块的开发。采用 JAVA 语言实现文档、图片、视频在线云存储预览系统；真正意义上解决了 ZIP 或 RAR 压缩包、图片文件、绝大多数音频视频、Flash 动画文件、文件存储、在线预览、下载以及在线解压缩一系列问题。本企业私有网盘系统能完全实现 FTP+移动办公业务，提高企业信息化办公管理效率。为企业搭建网络磁盘和企业文件的统一存储和管理平台，以及用户需求密切相关的私有云网络磁盘存储系统。同时为企业的所有员工创建一个快速高效的办公环境，并带有前瞻性的私有网盘云存储解决方案。该系统是一个高效稳定的内部私有云网络磁盘存储系统，具有使用更方便、功能更强大，数据更安全等功能。

本企业私有网盘系统在需求调研、设计开发及编码实现中充分考虑了用户实际业务使用场景的易用性、安全性、稳定性，并提供丰富的应用功能。实现了网络资源最大化共享，提高企业信息化数据交换和人性化管理的业务效率。

通过客户需求调研，系统设计不断满足企业应用程序要求，文件数据存储的核心代码逐渐形成。为了达到系统不会重复存储同一文件，系统在设计开发上采用先进的传输控制组件，然后启动第二个传递函数，将该文件添加到用户，同时该系统以解决网络用户数据存储、资源共享、文件管理等应用为需求。在考虑为用户提高数据应用的价值的同时，也最大限度地提高客户现有网络拓扑架构的便利性和共享性，更好地利用企业网络环境资源为企业的信息化建设提供可靠的保障和服务。

1.3.2 课题研究的主要内容

课题研究的主要内容是研究一种适应各种平台的私有网盘系统，通过科学的方法，对系统进行研究分析、设计系统架构及具体功能，并通过适当的技术选型进行系统的开发和测试，最终形成一套完整的系统软件产品。以下内容是针对该系统的研究：

(1) 针对课题所涉及的专业知识和技术进行研究，分析了项目的可行性，对课题当前国内外的研究状况和研究成果进行了学习和总结，拟定了课题研究的方法与研究内容。

(2) 在系统的开发设计过程中，我对系统所需的相关技术进行更全面的学习和研究。包括 JAVA 中面向对象的思想 and 软件 JAVAE 开发框架等技术，UML 建模语言里对如何实现面向对象的建模方法，程序应用的开发技术等专业知识的深入学习，为本企业私有网盘系统的设计和开发奠定了坚实的基础。

(3) 在客户需求调研过程中，我积极主动的开展现场业务需求研究，并与各

部门的相关业务人员进行了充分的交流操作和使用场景的需求分析。完成了本课题研究中系统参与者的需求分析、软件功能、安全性、可靠性和可扩展性，为软件设计和开发提供了可靠的依据。

(4) 在本企业私有网盘系统的设计和开发过程中，我充分利用了面向对象的系统设计和编程方法。在系统的架构设计、功能设计、类设计、对象设计、概要设计和详细设计等方面，我充分认识到 JAVA 是一种面向对象的编程语言，并熟练使用了 JavaEE 开发技术。

(5) 在认真分析和设计企业私有网盘系统开发要求的基础上，对项目完成的开发设计过程、系统功能和性能测试进行了充分的准备。对测试中的缺点进行了一些改进，形成了系统测试结论，完成了系统开发的主要任务。

1.4 论文结构

本文共分六章，阐述了私有网盘系统的研发过程。在使用软件工程的知识指定系统研发设计的基础上，通过对系统需求梳理和分析，归纳总结出一种基于分层次式的可行性开发模型。从系统需求分析、架构设计和系统编码实现以及系统单元测试集成测试等几个方面得出了研发结论。以下为论文各章节的主要内容：

在第一章讲述的绪论中，讲述了企业网盘系统研究的课题背景。国内国外市场价值和未来发展趋势提出了研究的目标和内容，本文的目录结构最后进行了阐述。

第二章讲述的是系统开发实现所需要的 JAVA 技术，其中包括面向对象的分析设计方法、UML 建模语言基于面向对象软件建模方法，JavaEE13 大开发技术框架。这些都是系统开发中涉及的主要理论方法和技术支撑。

第三章内容主要描述的是企业私有网盘系统在客户现场调研与业务需求分析的内容，基于持续与客户业务人员详细沟通交流的过程，阶段性的获取到了系统完整的需求文档，包括功能需求和非功能性需求。

第四章系统的设计编码与实现。给出了一个完整可行的敏捷开发持续迭代计划，包括了系统整体架构，模块开发实现的全过程。

第五章讲述的是企业私有网盘系统单元测试集成测试相关内容，介绍阐述了本系统的测试相关内容并根据单元集成测试得出来的结果，编写了系统测试用例和测试报告宏观上提出了系统整体测试的意义。

第六章是整体角度对网盘系统进行了需求分析、架构设计、编码实现、测试集成等，全流程开发总结和评价并对系统的优势进行分析和展望。

第二章 系统研发的技术原理

本章节是讲述关于私有网盘系统课题，系统前期的架构设计和具体的开发流程。在整个项目周期里使用到的 JAVA 基础知识和底层关键技术进行详细的讲解，其中细节有：JAVA 语言面向对象的程序设计和系统分析方法、利用 UML 统一建模语言进行软件模型设计，同时还大量使用了 JAVAEE 企业级开发中的很多核心技术知识。

2.1 JavaEE 系统设计开发

2.1.1 面向对象编程

万事万物皆对象。软件设计开发的重要思想指导和方法论述——面向对象编程。人类认识世界的惯性思维是面向对象的方法论，所以在软件的设计与开发中也要借助于这一面向对象的思维特性来辅助企业软件开发和产品设计。面向对象开发的好处：提高可重用性、可扩展性，可维护性。在 java 中，除了 8 个基本数据类型外，一切都是对象。java 语言不允许直接访问对象，只能通过对象的引用来操作对象。

（1）类

类是 Java 程序的最小单位。整个 java 程序是由一个个类组成，它代表了客观世界中的事物。从现实世界中的事物（对象）出发构造软件系统，在系统构造中运用人类思维方式，直接以现实世界中的事务（对象）为中心来思考，并根据这些事务的特点，把他抽象成为系统中的类作为系统的基本单元。使得软件系统的组织直接映射到客观世界。业务需要关心事物的属性状态，程序员就为这些状态定义成员变量。事务有哪些行为，程序员就为这些行为定义方法；类是具有共同属性，共同方法的一类事物。类是对象的抽象，对象是类的实例。因此我们要从使用类库的局限性思维中学会真正的面向对象的编程思想。

（2）封装

是面向对象实现软件复用的一个手段，当子类继承父类后，子类将直接获得父类的属性和方法。封装将对象的实现细节隐藏起来，通过一些共用方法来暴露该对象的功能，使用一些封装好的对象，调用对象的方法，设置对象的属性。

（3）多态

子类对象可以直接赋值给父类变量，但运行时依然表现出子类的行为特征，

即同一个类型的对象在执行同一个方法时，可能表现出多种行为特征。

综上所述，在现实世界中事物和事物存在关系，类和类之间就存在关系。例如：一般与特殊的关系、继承关系 `is a`、整体与部分关系、组合关系等。java 语言通过在一个类里保存另一个对象的引用实现组合关系 `has a`。

2.1.2 面向对象的概念与特征

（1）对象与类

对象是由描述其属性和执行某些操作数据组成的整体。类是一种特征相同相似的对象实体的抽象。作为对象创造的模型，它是静态定义。将现实世界的事物看作对象，将具有相同事物状态，相同事物行为的事物进行抽象，定义为类^[2]。现实世界中事物和事物之间存在关系，事物和事物之间有一般特殊关系，将这种关系抽象出来，表现为类与类之间的继承关系。事物和事物之间还有整体部分关系，将这种关系抽象出来，表现为类与类之间的组合关系。对象与类的关系如图 2-1 所示。

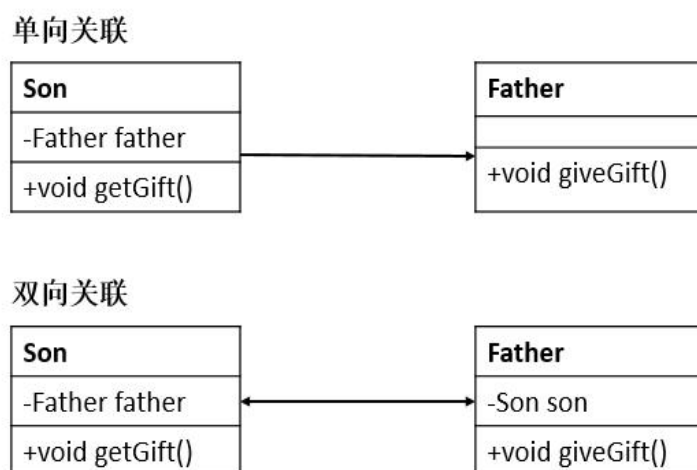


图 2-1 对象结构图

（2）继承

从宏观上讲，获取现有属性和特征是继承的特性。继承可以共享程序代码和数据逻辑，减少软件中大量的重复冗余代码。是面向对象实现软件复用的一个手段，当子类继承父类后，子类将直接获得父类的属性和方法。继承也具有可传递性。子类继承了父类的所有特性，继承机制的原理如图 2-2 所示。

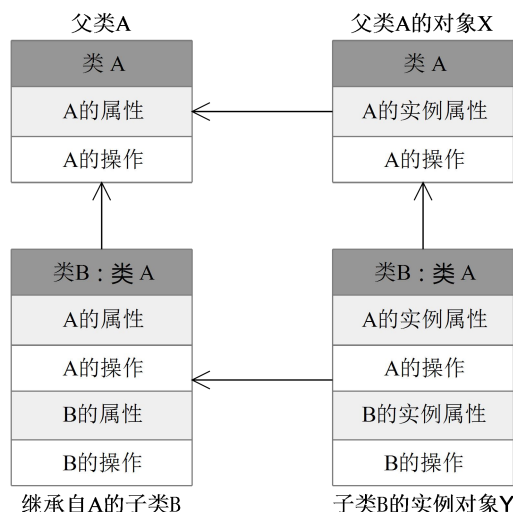


图 2-2 面向对象继承机制原理图

只有一个父类的继承方式叫着单继承，有多个父类的继承方式叫着多继承。这样的继承模式可以让面向对象的软件程序逻辑更清晰，同时也便于开发人员修改，运营维护和二次扩展。

（3）消息

消息是指对象之间的相互请求与协作。消息请求指源对象将消息实体发送到另一个目标对象。外部接口是指对象能够接受的消息和消息本身携带的参数的传递方式。

2.1.3 面向对象的分析

面向对象的分析思想方法可以用来分析和构建软件业务，对象建模+动态领域模型+功能模型的构建方式为面向对象设计架构与编程提供了坚实的理论支撑。需求梳理收集涵盖系统边界+系统业务功能+系统性能指标+系统部署运行环境描述，都是面向对象分析的应该涵盖的内容方向。

2.1.4 面向对象的设计

本设计中开发的企业私有网盘信息系统平台被梳理拆分为众多相对较小和简单的业务子系统并以降低系统逻辑复杂度和系统架构耦合度为目的系统架构设计。子系统与子系统之间，子系统与底层基础平台之间相互独立且松耦合联系。我们在实际开发工程中降低系统设计之间的耦合度，是系统设计中推荐的优秀设计标准。子系统间或模块间元素形成的紧密性特性，我们称之为内聚。软件设计开发过程中推荐开发人员编码时尽可能的高内聚为佳。

2.1.5 面向对象的编程

OOP 是一种面向对象的编程思想，通常我们也定义为面向对象编程。它是一种计算机编程体系程序设计，数据结构和方法逻辑的结合。面向对象是编程过程中的指导思想和方法论。

2.2 基于 UML 的软件建模

UML 中文名称（统一建模语言），它用语言和符号的手法图形化方式展示在软件工程中构建模型设计图纸上。UML 也是一种面向对象的建模工具，它具有可视化、文档化，图形化的特性，在应用程序的设计架构上具有广泛的使用价值。在并发类，分布式和嵌入式等软件系统设计中，也可以使用 UML 构建系统模型。

UML 是一种始于 1997 年的 OMG 标准。包括由需求分析到规格，再到构造和配置。面向对象的分析设计方法--UML，发展起源于 20 世纪 80 年代末并在 90 年代得以产生，并且被进一步发展最终成为公众接受的标准面向对象建模语言。

UML 具有简单统一、动静结合、可视化的特点，因此他是一种面向对象软件设计的标准化建模语言。在企业私有网盘系统的开发中，很多开发环节都使用到了 UML 统一建模语言来辅助研发设计。如下表 2-1 所示。

表 2-1 UML 表

种类	UML图	实际应用情况
结构性的UML (Structure Diagram)	类图 (Class Diagram)	分析业务概念
	对象图 (Object Diagram)	基本不使用
	构件图 (Component Diagram)	用来分析IT基础架构、软件架构等方面需求
	部署图 (Deployment Diagram)	
	包图 (Package Diagram)	很少使用
行为性的UML (Behavior Diagram)	活动图 (Activity Diagram)	使用至少其中一种图来分析业务流程，大部分情况下至少会用到其中两种图
	状态机图 (State Machine Diagram)	
	顺序图 (Sequence Diagram)	
	通信图 (Communication Diagram)	基本不使用
	用例图 (Use Case Diagram)	必会使用
	时序图 (Timing Diagram)	基本不使用

UML 建模语言的组件包括：类、对象、关联、职责、行为、接口、用例图、包、序列图，协作图和状态图等。它们能够进行标准的元素定义和表达，并支持面向对象。提供的集合供简单清楚确定模型表示元素。UML 具有可视化的强表达

力，支撑纷繁复杂的软件系统建模设计。软件系统在不同角度的投影，可以用图形收集梳理的方式来展示相关事物及其关系。这些图表由连接图展示，表示事物的顶点相互关系。

(1) 类图 (ClassDiagram) 在 UML 类图中，类由包含类名，字段和带分界线的的方法的矩形表示。比如下图表示一个 Employee 类，它包含 name, age 和 email 这 3 个属性，以及 modifyInfo()方法。关于软件系统在整个生命周期中有效的静态关系，可以通过类图来展示，它也是面向对象系统建模语言设计中最常见的模式。

(2) 对象图 (ObjectDiagram) 展现了一组对象以及它们之间的关系。类图的实例可以用对象图的方式展现，它用到与类图几乎完全相同的标签组件来描述。

(3) 用例图 (UseCaseDiagram) 用例图从用户日常业务操作使用场景出发描述了系统功能的静态用法，用于现场客户需求分析模型的创建。如图 2-3 所示。

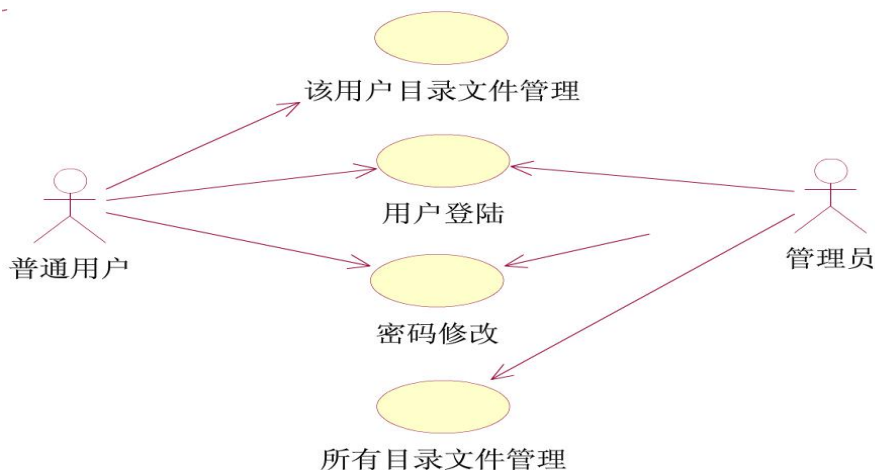


图 2-3 用例图

(4) 交互图 (Interactive graph) 通过对象之间的交互通信机制和消息传递机制，来阐述一组对象之间的动态关系。交互图又包括序列图和协作图，按时间顺序，逻辑先后关系组织的对象之间的交互活动称之为序列图，强调发送和接收消息的结构组织模型称之为协作图。

(5) 状态图 (StateDiagram) 状态图是类图的补充形式。

(6) 活动图 (ActiveDiagram) 它可以显示系统中一个活动到另一个活动的流转过程，活动图也是一种特殊的状态图。它有助于识别并行活动分析。

(7) 组件图 (ComponentDiagram) 部件之间的物理结构以及组件之间的依赖关系可以用组件图来表示。它有助于分析和理解组件与组件之间的关系。

(8) 部署图 (DeploymentDiagram) 是描述系统如何部署以及系统如何与其

他系统相关的图表。它通常描述一个节点、多个节点的构建部署关系。如图 2-4 所示。

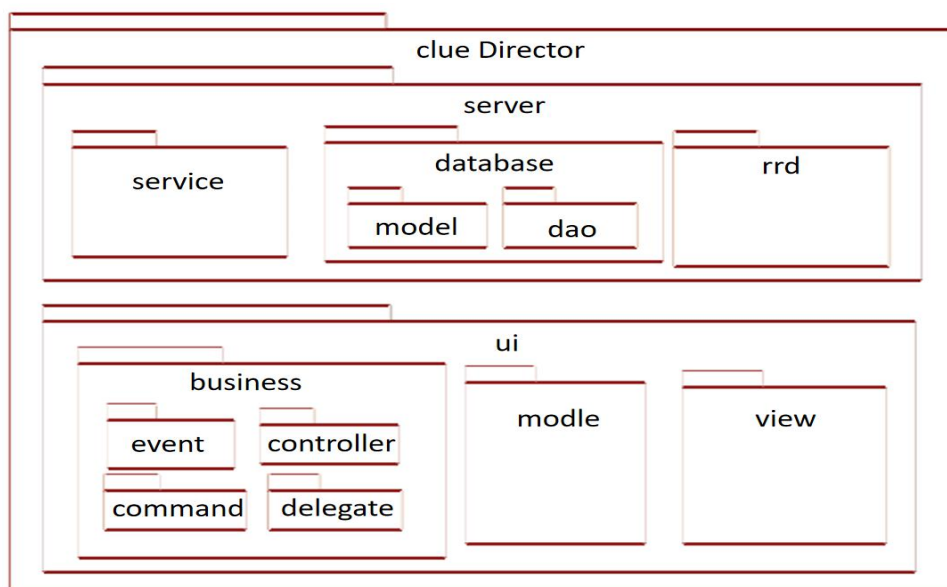


图 2-4 部署包图

2.3 JavaEE 开发技术

2.3.1 JDK

Java 语言的软件开发工具包（JDK），在它上面可以运行移动设备、嵌入式设备的各种 Java 应用程序，JVM 内存模型如图 2-5 所示。

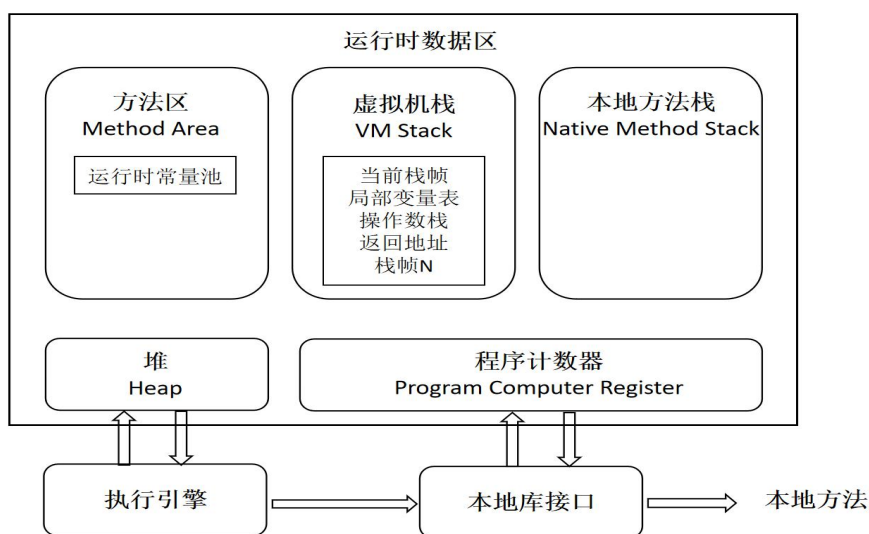


图 2-5 JDK 体系结构图

Java 是一种技术，它由四方面组成：Java 语言+Java 文件+JVM 虚拟机+应用程序 API。JVM 运行时环境即 Java 平台，开发人员编写 Java 代码 (*.java)，然后将它们编译为字节码 (*.class)，然后将字节码加载到内存中，一旦字节码进入虚拟机，解释器就会解释它。也可以通过即时代码生成器转换为机器代码执行。

(1) Java 源码编译流程图如图 2-6 所示。

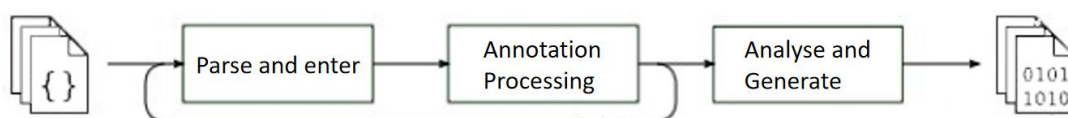


图 2-6 编译流程图

声明信息，域和方法声明信息，以及包含类/继承的超类/实现的接口的常量池。方法信息语句和表达式的信息在 *.java 源文件中包含字节码，异常处理程序表，评估堆栈和局部变量区域大小。

(2) 类加载通过 ClassLoader 及其子类实现，类加载顺序如图 2-7 所示。

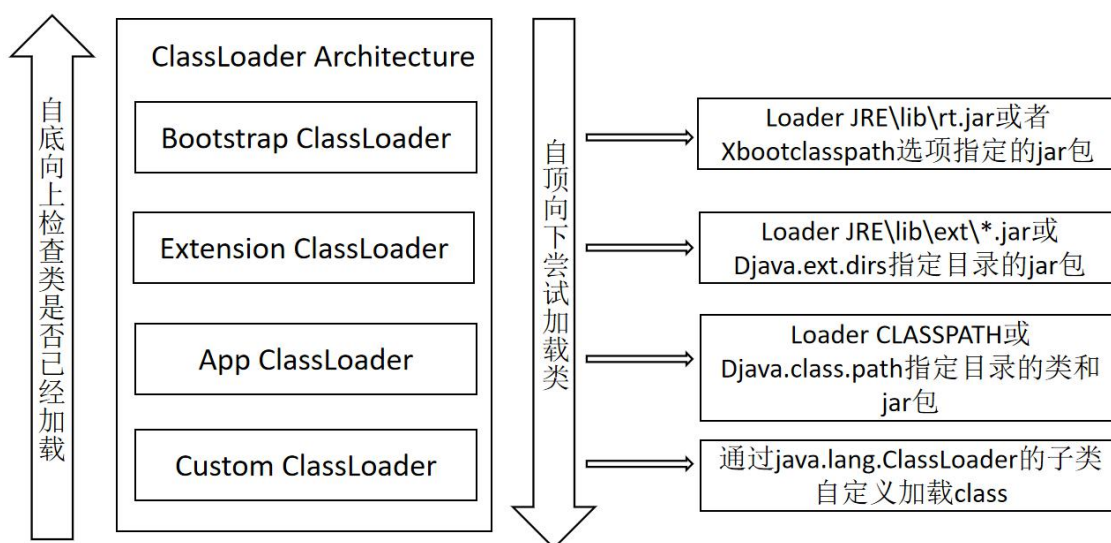


图 2-7 类加载图

(3) 类执行 JVM 是基于堆栈的虚拟机，每个新线程创建用到的堆栈通过 JVM 进行分配。堆栈以帧为单位保存线程的状态。JVM 对堆栈只进行两种操作：在帧中推送和弹出操作。操作数堆栈用于 JVM 虚拟机存储在执行方法期间生成的中间过渡结果。栈的结构如下图 2-8 所示。

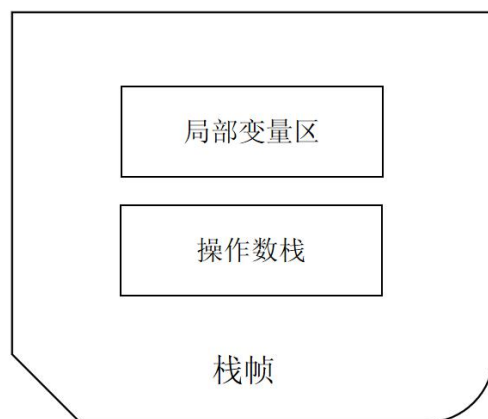


图 2-8 类执行图

2.3.2 JavaEE

(1) JavaEE 是分布式企业应用程序的开发规范标准。Java 语言平台有 3 个版本（小型设备和智能卡移动版 JavaME、桌面系统标准版 JavaSE、企业应用企业版 JavaEE）。

(2) 在 JavaEE 大家庭中，大量标准组件构成了 JavaEE 应用程序，我们把这些组件封装成相互独立的单元，它们通过相关的接口与其他组件交互。JavaEE 包括的主要技术有：Spring、Web Service、JavaMail、Struts、JSP、Servlet、JSF、EJB、JavaBean、JDBC、JNDI、XML、JavaSE、Hibernate 等等。

(3) JavaEE 拥有表示层/中间层/数据层这三层体系结构。数据层组件修改代码可以实现用户关于数据库查询数据需求的响应，如果用户提出了界面设计的变更需求也只需要修改表示层 UI 组件。由于表示层和数据层分层独立架构降低了冗余的垃圾重复代码，所以代码程序可以通过接口调用在三个层级通信传参，并且使众多的软件开发团队能够独立编写各自的组件部分发挥其团队优势。

2.4 本章小结

本章介绍了相关的软件开发知识理论和技术，涉及需求研究，系统设计和系统开发。面向对象的基本概念和特征、并充分使用 Java EE 平台相关技术来进行系统分析设计和编程；同时也充分使用 UML 统一建模语言来实现软件模型设计与研发。本章在进行相关技术分析的同时，对课题所研究和开发系统如何着手分析、设计和实现确定了技术方案，为后续系统研究与开发的过程形成基础。

第三章 需求分析

3.1 功能需求分析

企业私有网盘是采用 JAVA 语言实现的文档、图片，视频在线云存储预览系统。该私有云盘能够预览各种格式文件，其中包括 Microsoft Office 办公文档套件、金山 WPS 完整办公系列文档、开源 OpenOffice 办公系列文档、ZIP 或 RAR 压缩包、图片文件，绝大多数音频视频 Flash 动画文件。

本私有网盘系统可以为电子邮件附件、软件信息系统平台下其他所有子系统提供附件的在线预览服务（目前可以完美支持服务器端 Windows、Linux、OSX 等多种操作系统版本）。

3.1.1 统一存储服务

统一存储服务主要包含以下功能，具体服务功能如图 3-1 所示。

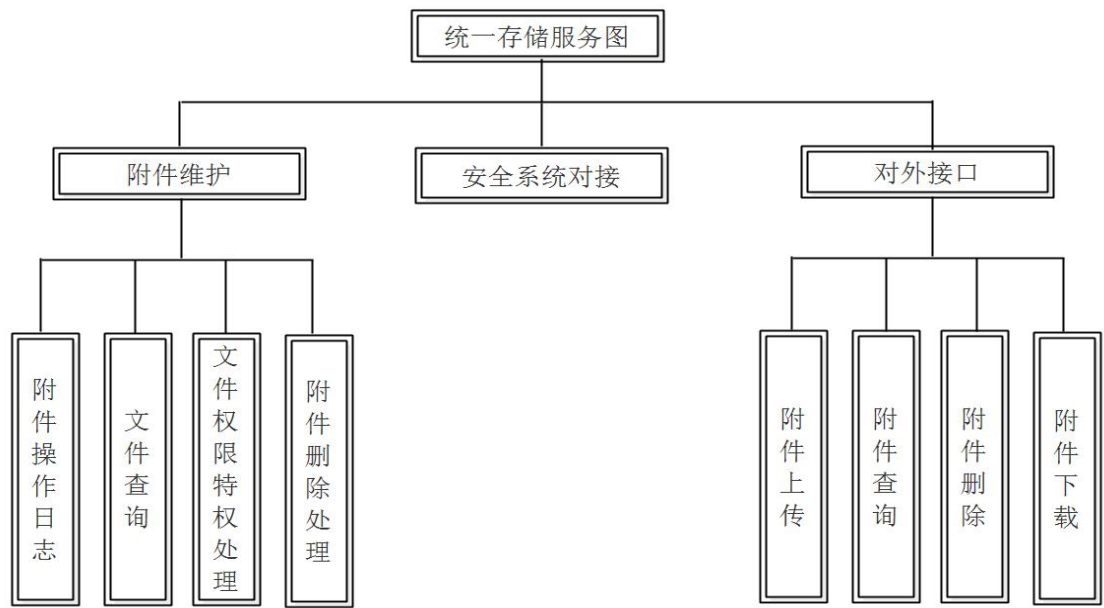


图 3-1 统一存储服务图

(1) 附件维护：

附件操作日志：调用统一日志服务，记录附件的上传、编辑、删除、下载、预览等信息；

文件查询：用户可查询系统中所有的文件记录（用户有权限看的），支持进行

快速检索：

文件权限特权处理：在特殊情况下，对文件进行了升密处理/修改知悉范围，这时候需要对该文件关联的所有附件进行权限调整，防止泄密；经过响应流程审批后，才能执行该操作。

附件删除处理：当删除文档附件后，需要把对应的附件记录 and 文件记录移到备份表中。如果文件没有被附件记录引用，那么需要把文件迁移到备份目

(2) 安全系统对接：

主要是调用安全存储系统的接口，进行文件的上传、更新、下载等操作，保证文件存储的安全。

(3) 对外接口：文件操作对外功能接口如 3-2 图所示。



图 3-2 文件操作对外功能接口

附件上传：接收其它服务/系统上传的附件，保存附件记录，并传输文件到安全系统中；

附件查询：按所属主文档、文件名称等进行查询；

附件删除：删除附件记录 and 对应的文件（遵循“附件删除处理”逻辑）；

附件下载：根据附件 ID 下载对应的附件。

采用 smb 协议进行连接，连接的时候需要经过认证，认证采用 NTLM 协议，对应的账号密码在集中配置中心进行配置。

3.1.2 文件视图浏览

本私有网盘系统提供压缩包或者文件夹等多文件视图浏览功能，非常类似于 Windows 的资源管理器界面，呈现清晰了然、操作简单。功能需求：地址栏多级导航条菜单、多文件夹及文件类型、图片缩略图+水印分页展示等需求。如图 3-3

所示。

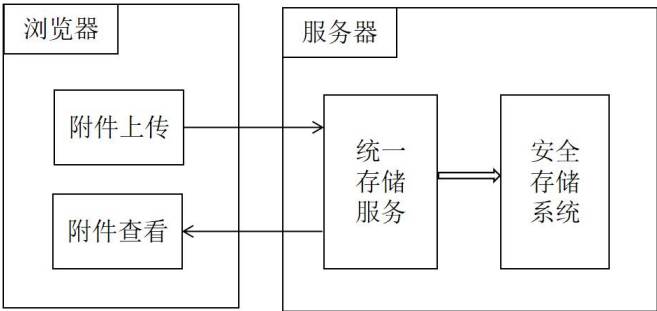


图 3-3 系统浏览需求分析

系统内其它服务如果涉及到附件的上传、下载等操作，需要调用统一存储服务的接口，进行附件信息的存储和获取。统一存储服务需要对接第三方安全存储系统，所有文件信息都会保存在安全存储系统中。本系统提供的功能主要是文档中附件的存储而来，涵盖附件信息的保存、文件的存储、附件权限的控制、系统之间附件的传输、大附件的分片处理等方面。由于本系统用于解决服务间、系统间的附件存储和传输，涉及数据的网络交互，需要考虑数据的安全性以及服务的可靠性、使用上的便捷性等方面的问题，故需求如下：

- (1) 保存文件分片 MD5 和分片大小的值，文件上传的时候通过 MD5 码以及文件大小判断文件是否已存在，实现秒传功能。
- (2) 在浏览器下载前，对文件大小进行检测，超过设定的阈值，提示用户采用客户端方式操作，但不强制。
- (3) 其它服务或系统只能通过统一存储服务进行文件操作，不能直接对接安全存储系统。
- (4) 大附件传输采用分片传输的方式进行。
- (5) 文件权限和文档权限分离，统一存储服务保存附件的权限信息，查询附件的时候要经过权限过滤。
- (6) 本部分权限控制，是由通过部署权限服务的机制，由权限机制拦截请求控制权限。

本私有网盘应用系统也借鉴了 Windows 特点使用简单，对于文件的管理方式类似于 Windows 资源管理器，整个网盘操作与本地盘的操作过程基本一致。不管用户年纪多大基本上在不需要培训的情况下，都能掌握网盘的使用

3.1.3 文档在线预览

支持 Word 、Excel、PPT、PDF、TXT、HTML 甚至是一些程序源码文件，都可以实现免下载在线预览功能。文档预览功能可以兼容市面上大多数在线阅读

文档格式。目前市场上同类产品的文件处理技术能力远高于市场，无需人工即可实现。文档在线预览具体业务流程描述如图 3-4 所示。

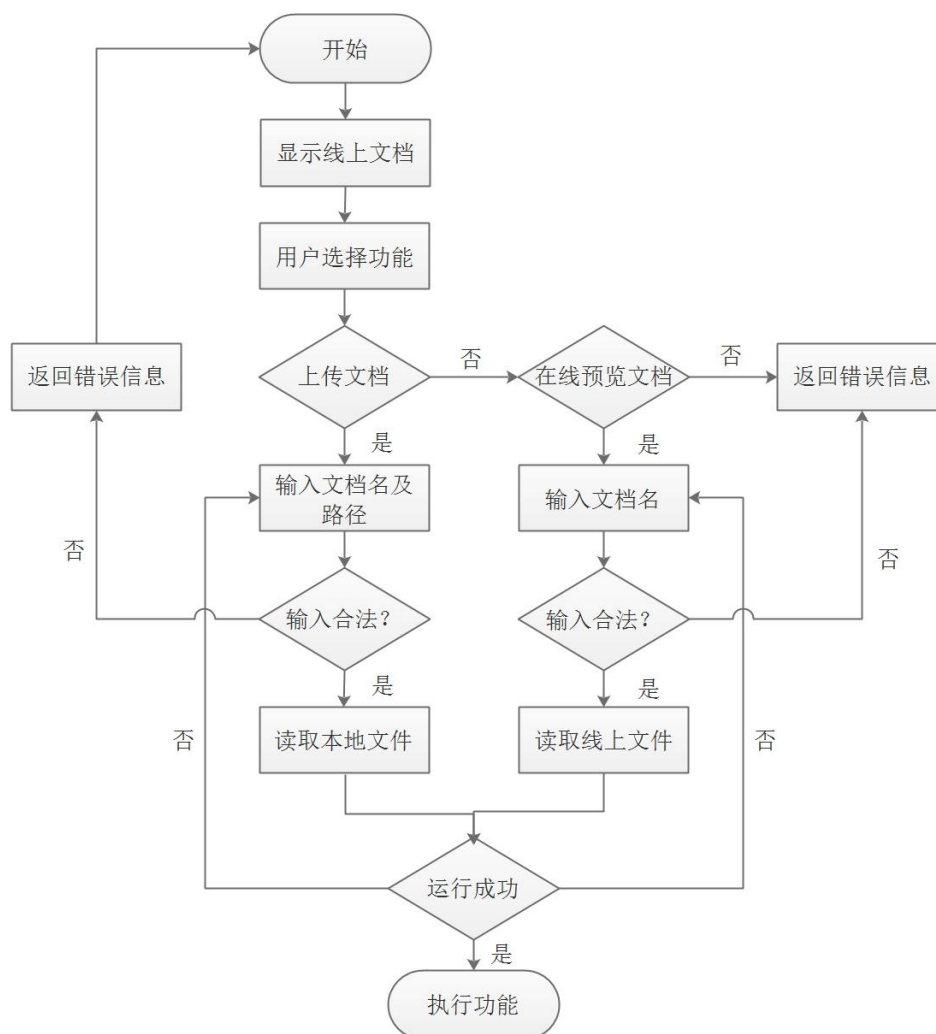


图 3-4 文档在线预览流程图

3.1.4 图片预览播放

私有网盘图片播放器，类似并强于 QQ 空间的照片浏览功能，多张缩略图同时浏览和原图可缩放大小、拖拽位置，图片自动播放功能。给用户非常震撼的视觉体验。其图片播放需求用例图如图 3-5 所示。

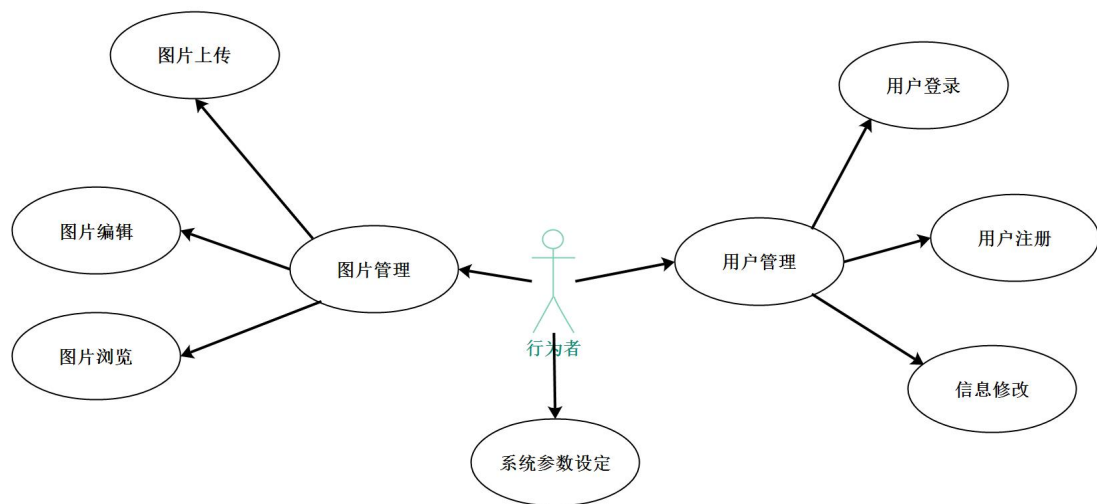


图 3-5 图片播放需求用例图

生成高清缩略图，添加图片、文字水印；图片、文字水印透明。具体需求如下：

（1）文档搜索列表展示，每个文件显示的图片大小为 100x100，缩略图生成是按比例缩小的，可能为 100x100、100x90、90x100、90x90，为了显示美观，我们需要让该图片在 100x100 的区域垂直、左右居中显示，那么就需要使用样式来控制，这时 WEB 前端开发人员建议，是否可以给图片加一个透明的背景，即将按比例生成的缩略图放置在一个大小为 100x100、背景透明的图片上。

（2）缩略图上可以添加水印支持（文字水印或图像水印）。

（3）需要实现文字、图片水印透明。

水印期望效果如图 3-6 所示。

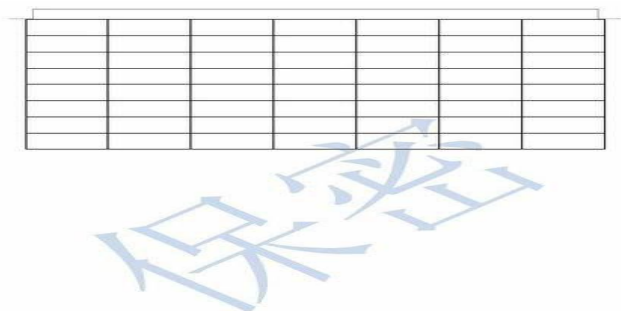


图 3-6 图片缩略图+水印效果图

3.1.5 音视频动画播放

企业私有网盘系统平台采用业界封装好的 ffmpeg 具有开源免费的特性，采用

硬件加速,更快的视频转码、裁剪、剪切、分离、合并、转换、拼接、水印、叠加、混合、转码、压缩、变速、添加 LOGO、添加滤镜、添加背景音乐、加速减速视频、倒放音视频、多个视频合成等工作。和普通的 `ffmpeg` 相比更快速。以兼容格式比较多的流媒体视频播放器。可以很清楚的听到 `wma`, `mp3` 等格式的动听音乐;可以很流畅播放 `mp4`, `flv`, `wmv`, `rmvb` 等众多格式的超清视频,还有闪动可爱的 Flash 网页动画。由于 HTTP 协议与 RTMP 等协议相比是无状态的。HTTP 具有更好的性能,但不适用传输过于实时的内容,因为延迟太大,性能不如 RTMP。如:视频聊天,电视直播等。

本系统音视频转码主要特点:

增加 H264 硬件编码器和 H264 硬件解码器,处理速度极大提升。增加 40 多个常见方法,编写各种辅助处理类。H264 硬件解码器免费,硬件加速编码器版本自动切换到 `libx264` 编码。其中 SDK 不访问网络,完全本地处理构造方法如果想扩展 `ffmpeg` 的命令可以继承这个类,然后像我们的各种 `executeXXX` 的使用一样来拼接 `ffmpeg` 的命令,不要直接修改代码文件,以方便以后的 `sdk` 更新升级。命令的字符串数组,可参考此文件中的各种方法举例来编写,新增在执行过程中取消的方法。如果在执行中调用了这个方法,则会直接终止当前的操作,此方法仅仅是在 `ffmpeg` 线程中设置一个标志位,当前这一帧处理完毕后,会检测到这个标志位,从而退出。因为 `execute` 是阻塞执行,你可以判断 `execute` 有没有执行完来确定程序是否完成。把一张图片变成视频形成视频的总时长,返回处理后的视频。包括如下内容:两个 `pcm` 格式的音频数据,(裸数据)混合 `pcm` 格式的主音频、主音频采样率、主音频通道数、`pcm` 格式的次音频、次音频采样率、次音频通道数、主音频的音量、次音频的音量。输出文件输出也是 `pcm` 格式的音频文件,把 `pcm` 格式的音频文件编码成 AAC,需要源 `pcm` 文件、`pcm` 的采样率、`pcm` 的通道数和输出的 `m4a` 合成后的文件。把 `pcm` 和视频文件合并在一起,`pcm` 数据会编码成 `aac` 格式,要注意的是需要原视频文件里没有音频部分,如果有则需要先需删除后,在输入到这里。

有时,我们采访的视频可能需要限制。例如防盗链,视频费用等。采用 HTTP 协议的问题是,一旦用户访问视频就可以将视频下载用于其他目的。通过分段进一步增加负载容量为了在保持更少连接的同时为更多用户提供服务。节省网站流量和服务器资源。当用户打开视频时只读取部分视频,如果视频未被分段则用户在用户打开网站时立即加载所有视频数据这极大地浪费了流量。在对视频进行细分后,我们可以将视频加载一段时间,以便我们可以加载尽可能多的视频。这样的处理方式虽然到达效果确浪费了流量,视频分片图如图 3-7 所示。

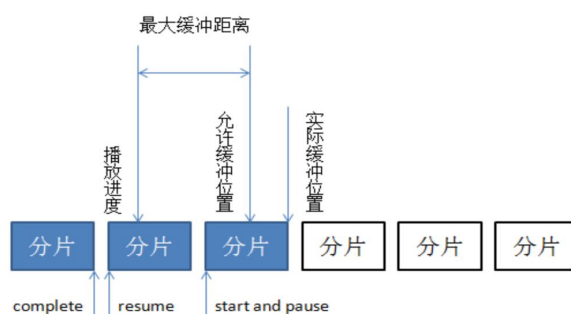


图 3-7 视频分片图

本系统音视频转码需求分析需要考虑以下几个方面的问题：

(1) 一条命令即可完成——阻塞执行：

```
VideoEditor editor=VideoEditor();
```

需求分析点 a:

视频增加水印:"处理后的视频"=`editor.executeOverLayVideoFrame("视频路径", "增加的图片路径", "x 坐标", "y 坐标");`

需求分析点 b:

裁剪视频时长:`dstResult=editor.executeCutVideo("视频路径", "开始时间 S", "结束时间 S");`

需求分析点 c:

画面裁剪:`dstResult=editor.executeCropVideoFrame("视频路径", "x 坐标", "y 坐标", "裁剪宽度", "裁剪高度");`

(2) 系统转码功能需求有替换背景音乐，声音混合和多段视频的拼接。可以进行不同视频源的拼接、视频画面拼接、读取视频帧、获取所有帧、图片转视频、倒序、加减速、镜像、音频混合、音频转码、画面填充、缩放、压缩、镜像、增加文字、视频转码，图片视频转 GIF 等操作。

(3) 如果列举的功能不能满足用户的需求时，用户可以采用自定义的方式根据 `ffmpeg` 的命令自行扩展，在代码接口中加以说明。

视频转码通过调整视频的 `bitrate` 来对视频文件大小进行压缩，降低视频文件的大小。但压缩可能导致视频画质下降、源视频压缩百分比值从 0--1 分离 `mp4` 文件中的音频，并返回音频的路径；音频裁剪需截取音频文件中的一段，尽量保持裁剪文件的后缀名和源音频的后缀名一致；剪切 `mp4` 文件（包括视频文件中的音频部分和视频部分）即把 `mp4` 文件中的一段剪切成独立的一个视频文件。此方法裁剪不是精确裁剪，而是从视频的 IDR 帧开始裁剪的，并没有精确到指定的那一帧的时间。如果指定的时间不是 IDR 帧上的时间，则退后到上一个 IDR 帧开始。

精确裁剪的同时缩放到指定位置。这个可以设置宽度和高度，其中宽度和高度是采用软缩放的形式来完成。要缩放到的宽度建议是 16 的倍数。要缩放到的高度建议是 16 的倍数。

FLV 文件格式全称（Flash Video）是一种流媒体的新视频文件格式。它得到了 Flash MX2004 的完美支持。FLV 是 Macromedia 开发的流媒体视频格式。这种格式是基于 Sorenson [20] 的压缩算法开发的。FLV 压缩和转换非常方便，适用于短片。一般 FLV 流媒体文件包位于 SWF 播放器的外壳中，FLV 可以很好地保护原始地址并且下载以保护版权。FLV 网页在线播放流程如图 3-8 所示。

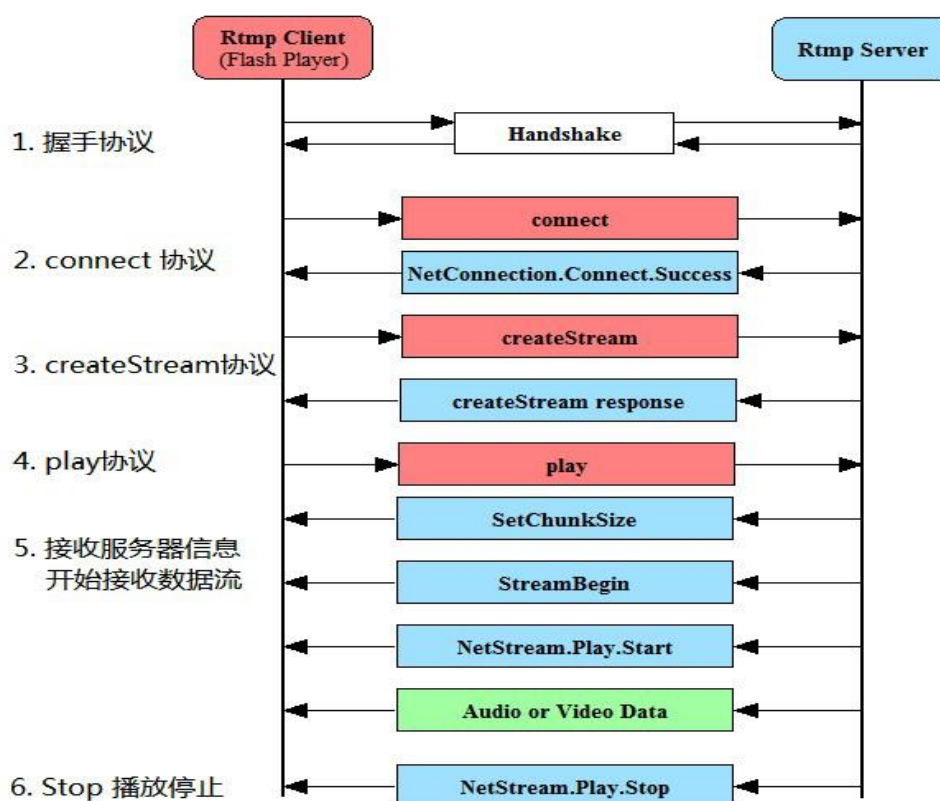


图 3-8 流媒体在线播放协议

为了解决 Flash 端播放视频在 HTTP 视频播放的方式，NetStream 提供有两种（普通模式和数据生成模式）：在普通模式下 NetStream 将会自动加载视频文件并开始播放。HTTP 视频也可以任意暂停播放，但却无法暂停视频数据的加载。在数据依然加载完成后，如果要对视频进行分段，然后在每个视频片段中来回随机点击，则必须制作与 NetStream 实例对应的分段视频。在此时，视频文件数据就可以由 URLStream 对象工具类来进行加载播放。但要注意，不要将所有数据都放入内存中否则内存会引发爆炸现象。具体的缓存策略后续具体讲。与大多数视频网

站平台上的其他流媒体视频播放器不同的是，Flash 播放器无法直接访问本地文件，但它可以通过加载已完成的视频文件并从缓存中快速获取视频数据流。所以如何有效利用缓存是优化的关键。

在普通传统分段视频播放方式下，我们可以直接来播放单个视频文件又像播放完整文件一样播放分段后的视频文件块。由于时间关系本企业网盘程序中设计实现的有点遗憾，后续利用空余时间会基于此解决方案进行深入优化。在视频开始播放之前，播放程序将获得视频片段的播放开始时间、播放结束时间和播放分段地址以及总视频元数据信息的列表。如图 3-9 所示。

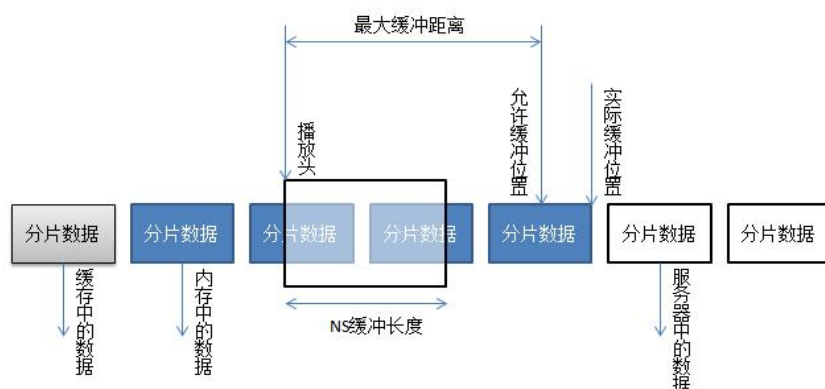


图 3-9 视频分片+缓冲图

3.1.6 在线解压缩

采用的是 JAVA 的压缩包工具类 ExtZipEntry 和 ZipFileEntryInputStream 能够对主流 zip、rar、tar、gz 等压缩文件格式进行完美解压。并采用了 JAVA 递归代码编写技术，对多级压缩包进行逐一全部解压缩。对于有密码的压缩包文件系统采用了 AesZipFileDecrypter 工具类通过 AES 方式解密并解压文件。对解压缩之后的文档文件采用 JAVA 多线程技术，事件驱动模型在服务器闲置的时候调用文档转换组件进行用户预览事前转换。提升服务器文档预览的相应速度在线解压缩具体功能需求如下：

实现对目标路径及其子路径下的所有文件及空目录的压缩与解压（单个文件、文件夹以及文件和文件夹的组合压缩与解压）。

支持中文文件目录与名，解决乱码问题，支持无限层级自动解压。

将多个文件压缩与解压后，提供网盘里压缩包文件在线预览与下载的需求。

具体业务逻辑线图如图 3-10 所示。

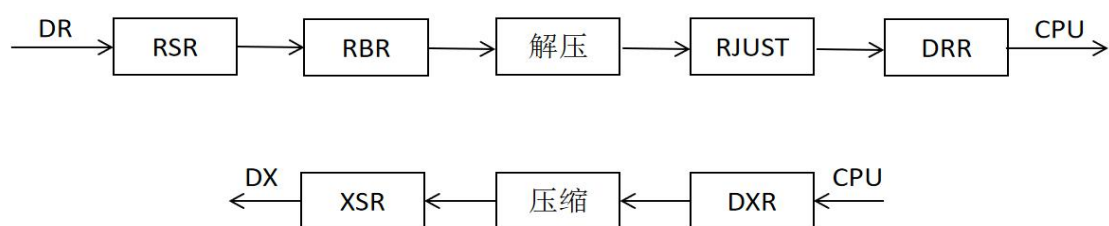


图 3-10 数据压缩解压缩逻辑线路

3.1.7 文件上传下载

类似于百度图书馆，百度网络磁盘在线文档管理系统。需求支持文件拖放上传+文件上传前预览详细信息+多文件上传列表显示+超大文件上传等。其上传系统部署需求图如图 3-11 所示。

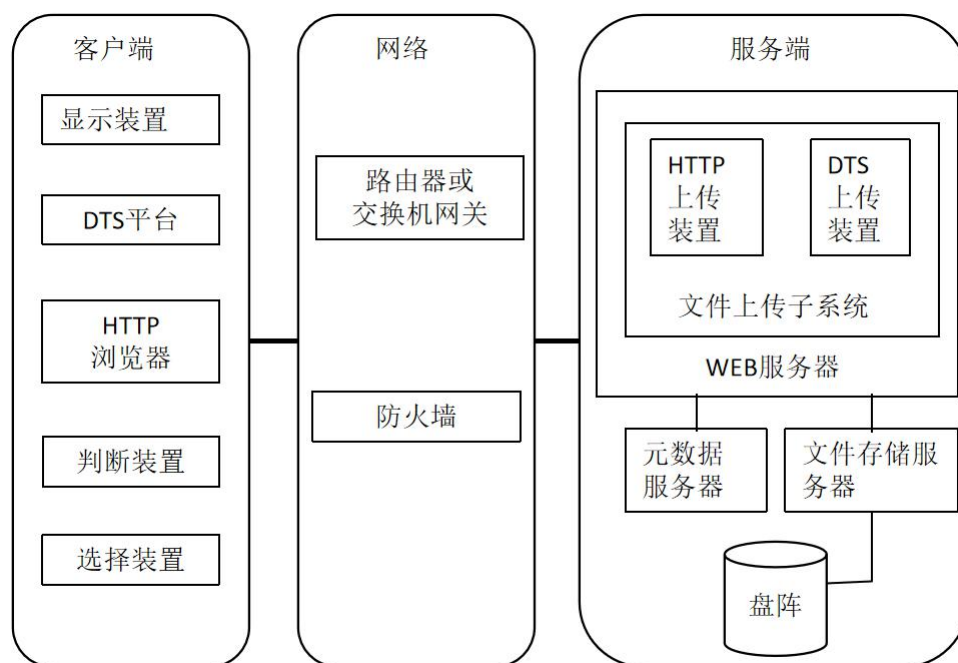


图 3-11 文件上传系统部署需求图

文件上传具体功能需求如下：

- (1) 拖拽上传文件。
- (2) 超大文件分片上传。
- (3) 文件上传下载有进度条显示并支持断点续传。
- (4) 带有中文名的文件上传时防止出现乱码。
- (5) 文件上传的格式可以多种类，但是要过滤可执行文件上传进行限制。
- (6) 支持文件按照目录结构上传和多文件批量上传，智能拆分文件分类处理

如图 3-12 所示。

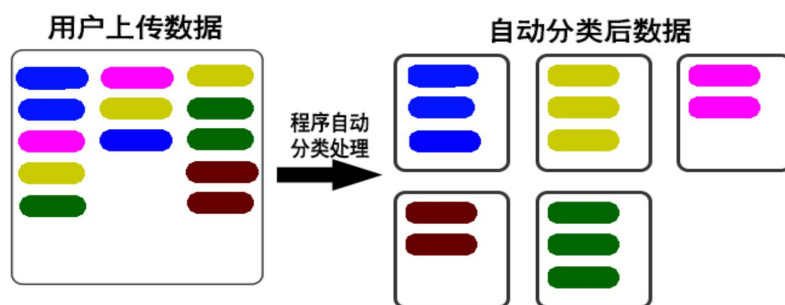


图 3-12 智能文件分类处理

文件下载具体功能需求如下：

- (1) 支持本地下载并保存或另存为。
- (2) 在下载页面进行下载，支持浏览器文件头信息的读取下载完成后匹配对应程序软件提示打开使用。
- (3) 显示下载进度，并支持大文件断点下载。

3.2 非功能需求分析

私有网盘系统需要实现的功能性需求大致包括：支持复杂文件预览、精准高品质文件转换、支持超大文件转换、极速分页加载、无限极快速解压缩。

3.2.1 支持复杂文件预览

保证普通、复杂文档 100%能在线预览。采用阶梯式优先级文档转换原则：

MS-Office-->>金山 WPS-->>OpenOffice-->>PdfCreator-->>SWFTools 转换路线。能够兼容轻松转换普通、复杂文档。像复杂表格、数组签名、电子签章等都能保证 100%能够转换预览成功。

解决用户如下问题需求：转换的 html 网页对复杂的文档、表格、PPT、电子签章转换出来几乎完全变形，或者有的根本不能转换。

3.2.2 精准高品质文件转换

使用 MS Office、金山 WPS、OpenOffice 三大办公自动化应用软件，100%确保文档转换成功，并保证转换后的准确性与实现质量高清逼真效果。

解决用户如下问题需求：换出来的文档全部是普清版本，如果要高清版本需要另行支付昂贵的费用。并且所谓的付费高清版仍然不能保证 100%文档转换的精准性，达到高清逼真的效果。

3.2.3 支持超大文件转换

经过反复 100MB 以上的超大文档压力测试，都能够稳定的转换成功。目前很多同行文档处理中间件都不能完全达到此功能。

解决用户如下问题需求：在实际使用过程中以前能够转换预览的文档都非常小。对一些“大块头文件”简直是束手无策。

3.2.4 极速分页加载

实测中在线预览过上千页的电子文档、长篇小说。通过模仿百度云盘分页加载的功能，达到极速的网络传输效果。

解决用户如下问题需求：无论文档有多大，页数有多少全部采用的是无分页一次性加载。这就直接导致浏览速度缓慢，如果遇到文档过多过大时，根本无法加载文档内容，或加载中导致浏览器死机的问题。

3.2.5 无限极快速解压缩

支持.zip、.rar 等主流压缩文件的在线解压缩（一些提供密码的压缩文件也可以）。支持的文档类型非常丰富如：word、excel、ppt、html、xml、txt、源代码文件等，并自动装载有大量的字体库文件保证转换后无乱码现象；有一款非常酷炫的图片播放器放大、拖拽、自动播放、分组切换无所不能；支持多种音视频文件的在线播放。

解决用户如下问题需求：传统模式下，文档转换服务器支持转换的文档类型非常有限，文档浏览界面也非常简单。在线解压缩文档也根本无法支持。

3.2.6 安全性、可靠性、可扩展性

（1）安全性：该企业私有网盘系统是一个开放的多用户角色的 Web 应用系统，需要具有良好的稳定性与安全性。具备用户身份、角色验证，关键数据操作保护，日志跟踪机制，关键操作权限判断等安全性措施。

（2）可靠性：系统需要具备一定的容错机制。对于用户输入有简单容错措施，系统有一定的容错屏蔽功能，系统具备基本的容错恢复措施。系统能够在主流设备运行环境中正常运行。

（3）可扩展性：系统设计与编码考虑后续功能扩展。企业网盘系统架构部署采用分布式服务器扩展，增加了系统运行维护数据监控界面。

3.3 本章小结

本章节主要内容基于面向对象的系统分析与设计，使用 UML 建模工具进行系统设计建模。首先根据企业私有网盘系统的总体需求情况明确系统需要解决的问题域，明确系统边界；分析系统的参与者，根据需求中对参与者的定义进行参与者识别与建模；对用户网盘使用过程中的业务工作流程进行分析梳理，以用例的方式进行功能描述与分析，完成了系统功能的整理与初步分析任务。描述了系统安全性方面的要求，可靠性方面的要求和可扩展性方面的要求。

本章描述需求收集整理分析方面的工作，需求分析的任务采用面向对象的分析设计和架构建模方式完成。为后续的系统设计和实现部分打下基础，本章中的结论和数据模型及功能界面原型将作为下一阶段工作的基础和指导。

第四章 系统架构设计与实现

参考需求分析得出的结论，本章节采用面向对象程序设计的思想方法。充分考虑系统的安全性、实用性、可扩展性和可靠性来设计系统架构、业务功能，数据库结构，并用特定技术实现系统开发。

4.1 系统架构

企业私有网盘系统是采用 JAVA 语言实现的文档、图片，视频在线预览系统。JAVA 语言本身具有先天跨平台跨系统的优势特性，因此本企业网盘系统可以完美支持多种版本操作系统上应用程序的运行，例如服务器端 Windows，Linux 和 OS X。

除本系统 APP 工程文件之外，还要用到的第三方软件有微软 Office2010 及以上版本、金山 WPS2015 及以上版本、开源 OpenOffice 4.0.1 及以上版本、PdfCreator 软件（另一种文档转换方案可选安装）、SWFTools -2013-04-09-1007 及以上版本、JDK1.6 及以上版本、Tomcat6 及以上版本。整体架构图如图 4-1 所示。

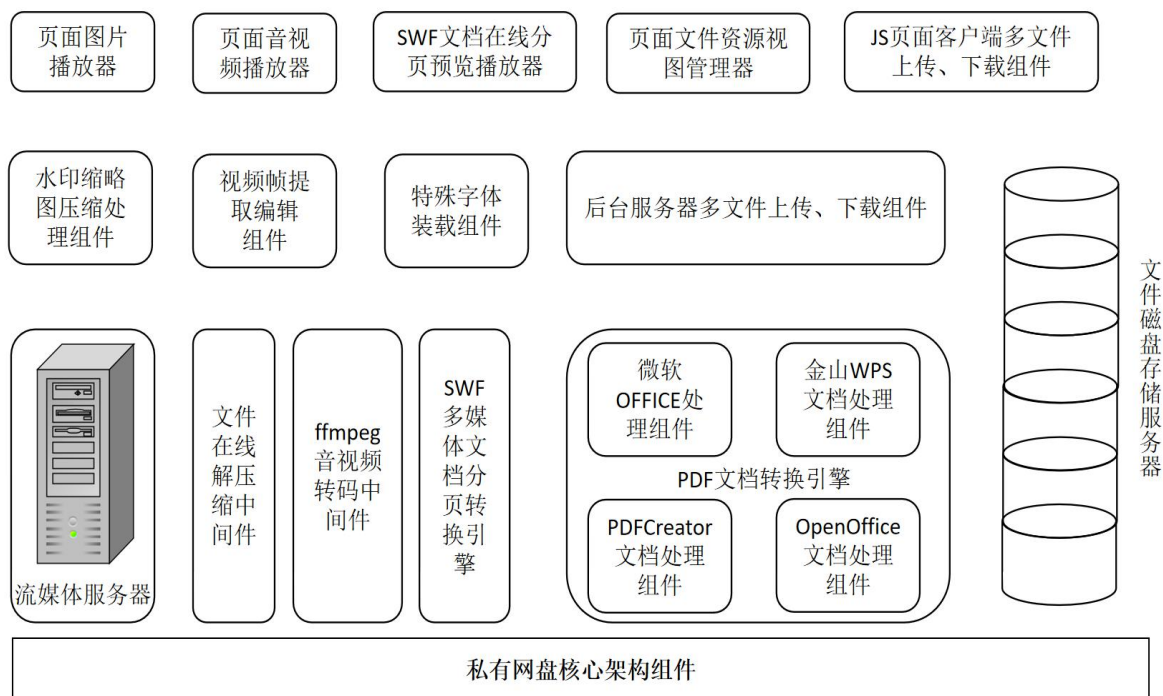


图 4-1 系统整体架构图

系统采用分层架构设计，分为表示 UI 层、业务逻辑 BLL 层。业务逻辑层通过接口和业务参数处理组件实现系统业务处理任务。通过多层结构的设计，可以提高程序的可重用性，提高系统的开发效率，实现高内聚和低耦合的效果。第三方文档转换软件使用阶梯式优先级文档转换原则：MS-Office-->> 金山 WPS-->>OpenOffice-->>PdfCreator-->>SWFTools 转换路线。

4.2 系统详细设计

4.2.1 系统组件设计

系统总体功能由 13 大核心组件和中间件完成实现，系统总体服务架构如图 4-2 所示。

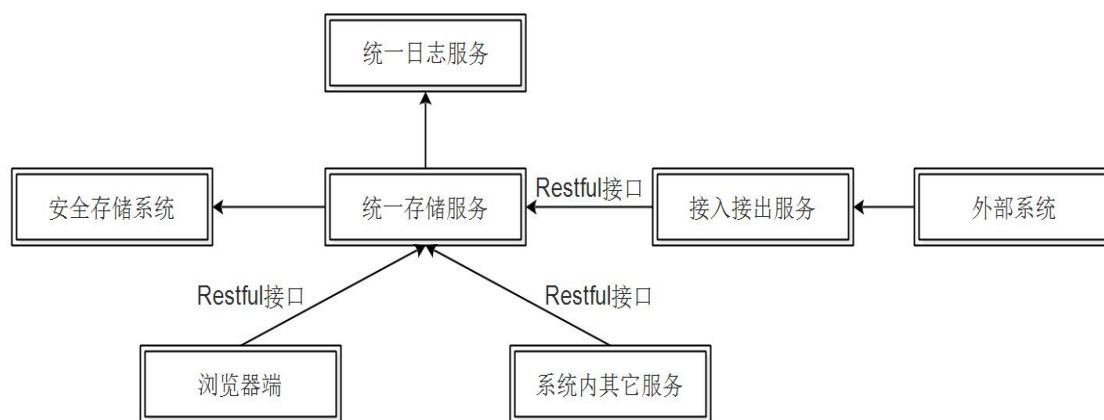


图 4-2 系统总体服务架构

统一存储服务涉及到文件数据的传输，考虑到传输的效率和稳定性，接口统一使用 http 协议。浏览器端，系统内的其它服务都通过 RestFul 接口进行附件的操作。外部系统附件操作需要经过接入接出服务。系统内的其它服务可以通过 RestFul 接口调用统一存储服务，进行附件的上传、下载、更新、删除；统一存储服务调用统一日志服务保存附件的操作记录；统一存储服务调用安全存储系统的接口进行附件的存储和传输。

（1）多文档上传组件

采用的是 JQuery+Bootstrap 的前端 JS 框架的 FileInput 插件实现的多文件上传效果，具备文档文件上传前的预览功能；上传时对文件的添加移除，文件类型、大小合法性校验；并能够显示多文件上传进度条。文件存储架构如图 4-3 所示。

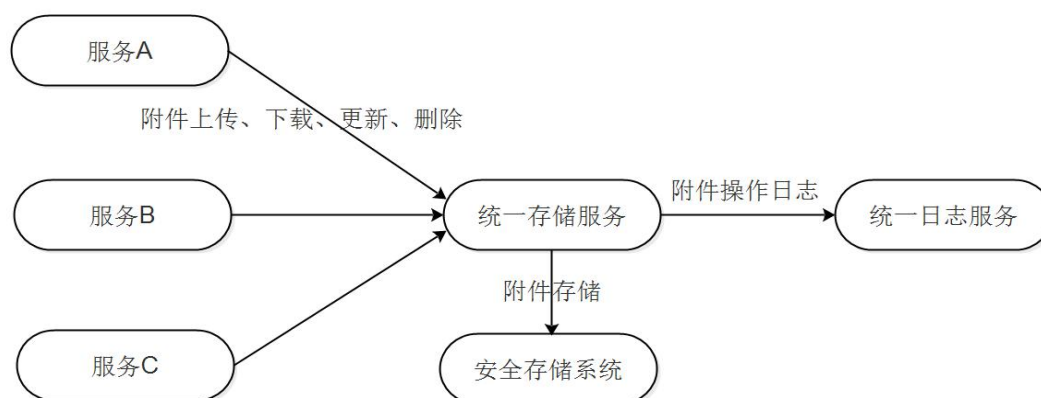


图 4-3 文件存储服务架构

（2）文档类型识别器

采用 JAVA 的文件文件操作工具 File 类，通过动态获取文档文件后缀扩展名的方式，然后跟私有网盘系统中缓存的 FileType 枚举类进行自动匹配。最后在利用文件适配器的方式进行分发到对应的文档转换引擎或者文件分类磁盘存储服务器。

（3）PDF 文档转换引擎

该引擎是整个专用网络磁盘系统的核心组件。为了保证稳定转换的两个 100% 指标，即：（100%能成功转换且 100%能够与源文档保持格式一致）。PDF 文档转换引擎采用 Jacob 技术在 Window 下调用 DLL 动态链接库实现文档的转换。通过命令行指令的形式与微软 Office、金山 WPS、PdfCeator 通用标准文档处理软件 API 接口通信的方式来进行文档的 PDF 转换工作。私有网盘引擎是非常具有稳定性和健壮性，它会智能的识别当前服务器上所安装的软件环境，如果不具备上面提到的 Office 等办公文档处理软件或者调用这些软件时发生异常，那么引擎会自动的切换新的文档转换方案。在开源 Jodconverter 工具包中使用 JAVA 网络通信套接字套接字连接，通过 IP 服务端口的形式去调用 Apache 开源组织提供的 OpenOffice 软件来进行文档的转换工作，其中使用到了 JVM 虚拟机 xpdf 字体库动态装载，解决不同操作系统下的文档乱码问题。并且开发架构设计中还预埋了 POI、Itext 等开源文档处理框架，未来对文档进行在线编辑处理，如：“Word 文档文件头套红”等。为了提高转换的性能，系统还采用了多线程的编程方式，并且做好了相应的多用户下的并发控制。

（4）SWF 多媒体文档分页加载引擎

考虑到当我们在 B/S 浏览器客户端上面进行文档的在线阅览时，PDF 格式的文档是需要安装阅读插件的，绝大多数非专业计算机用户是没有安装这些插件

的，并且在不同的系统下安装容易报错，用户安装成本太高。因此为了用户体验更好，本系统采用了 SWF 分页文档转换程序。这样用户不用关心安装复杂的插件，一般浏览器都普遍安装了 FlashPlayer 播放器。通过借用这一原理，就可以对我们 SWF 格式的文档进行原样播放。同时我们做到了分页加载，在服务器端将文档按页码切割成块，并保存成*.fubo 的文件块后缀名。这样同时带来了另外一个区别于同行竞品的明显优势——轻松支持在线阅览超大文件、千页文档。

（5）文档在线分页预览播放器

采用的是在开源软件 FlexPaper 基础上完全重构、深度定制具有特色的一款 LibraryPaper.swf 播放器，该播放器体积非常的小只有 100Kb 左右，并且拥有自己的企业标识 Logo 和知识产权，能够快速在网络上传输加载、分页播放后缀名为 *.fubo 的 SWF 多媒体文档，也能兼容 PC、移动终端的各版本浏览器。同时该播放器还具备了一下常用的文档操作功能比如：打印、查找、放大，拖拽等。

（6）文件在线解压缩中间件

采用的是 JAVA 的压缩包工具类 ExtZipEntry 和 ZipFileEntryInputStream 能够对主流 zip、rar、tar、gz 等压缩文件格式进行完美解压。并采用了 JAVA 递归代码编写技术，对多级压缩包进行逐一全部解压缩。对于有密码的压缩包文件系统采用了 AesZipFileDecrypter 工具类通过 AES 方式解密并解压文件。对解压缩之后的文档文件采用 JAVA 多线程技术，事件驱动模型在服务器闲置的时候调用文档转换组件进行用户预览事前转换。提升服务器文档预览的相应速度。

（7）私有网盘基础核心架构组件

主要采用 JAVA 核心技术，例如：文件流 IO 操作的工具类组件、XML 文档结构的解析、图形图像的操作与转换、Socket 网络套接字通信、日志的处理、操作系统 API 调用、多线程编程、日期工具类、集合工具类、Web 编程等；同时还使用了很多第三方开源代码工具包。

（8）文件磁盘云存储服务

在云存储方面采用了 Hadoop 云计算、Mongodb 文档数据库、Redis 分布式缓存、CDN 加速、Nginx 静态服务器负载均衡等多种技术。

（9）流媒体服务器

企业网盘流媒体视频播放程序主要使用 Red5 开源代码深度修改的定制化流媒体服务器。本系统中主要使用该技术来进行音视频的播放功能，并没有涉及实时聊天系统相关的技术。视频切片完成后播放不要急于将其关闭，可能需要保留以供后续搜索。在下一个切片上执行 resume 方法。寻找视频加载播放切片的卸载部分，关闭视频切片正在加载的视频流，并使用 NetStream 将它碎片化的从搜索

位置指定的位置重新加载（指定 start 开始播放参数）。当连接池数量满的时候，还可以从连接池中获取新的 NetStream 程序，它将自动关闭一些旧的 NetStream 连接。此连接消除原理原于空间局部性，应当首先关闭距当前播放视频头最远的切片（最大缓冲内的切片不能被关闭）。因为根据概率统计，大多数搜索是在播放头附近找到的。请求完成后，可以从浏览器缓冲区获取对同一资源的下一个请求，所以小切片更容易被缓存。由上面的小切片引起的问题不再存在。可以像请求服务器数据一样快速请求此数据。白框是尚未加载的数据，它们正等待加载到服务器上。为了保证搜索的准确性，而且服务器实现也非常简单。非常推荐这种方式。数据的三级查询如图 4-4 所示。

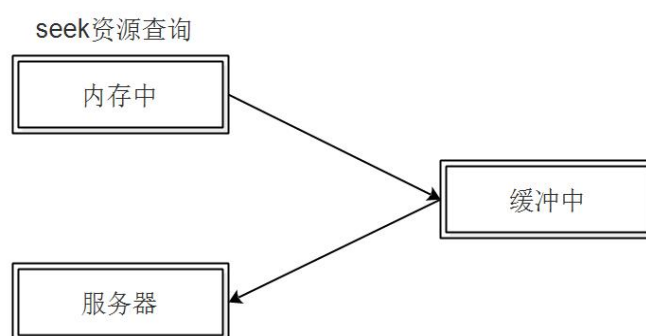


图 4-4 流媒体视频三级查询图

（10）音视频转码引擎

为了便于音视频文件在网络间传输与播放，我们将所有音视频文件分别统一转码成了 MP3 和 FLV 两种主流媒体格式。编写转码中间件主要采用了 ffmpeg 音视频处理软件，通过 JAVA 代码自动基于命令行的调用方式，它还实现了对 Windows 操作系统、Linux 跨操作系统平台的转码服务器支持。转码之后通过提取视频第一帧图像来动态直观生成视频的页面预览图片。

FFMPEG 转解码核心代码：

视频内容容器格式注册: registerAll(...)

打开打开输入视频文件: openInputFile(...)

从视频流文件中提取流头信息: findStreamInfo(...)

定义所有视频流查找其种类: codecTypeVideo(...)

匹配对应的视频流解码器: codecFindDecoder(...)

创建视频编解码器工具类: codecOpen(...)

程序为视频流解码帧分配足够的内存: codecAllocFrame(...)

解码程序连续从视频码流中抽取出相应的帧数据: readFrame(...)

判断视频抓取帧的图片类型对于视频帧进行解析: `codecDecodeVideo(...)`

视频编解码完释放解码器内存占用: `codecClose(...)`

关闭结束视频流输入文件: `formatCloseInputFile(...)`

主要数据结构, 如图 4-5 所示:

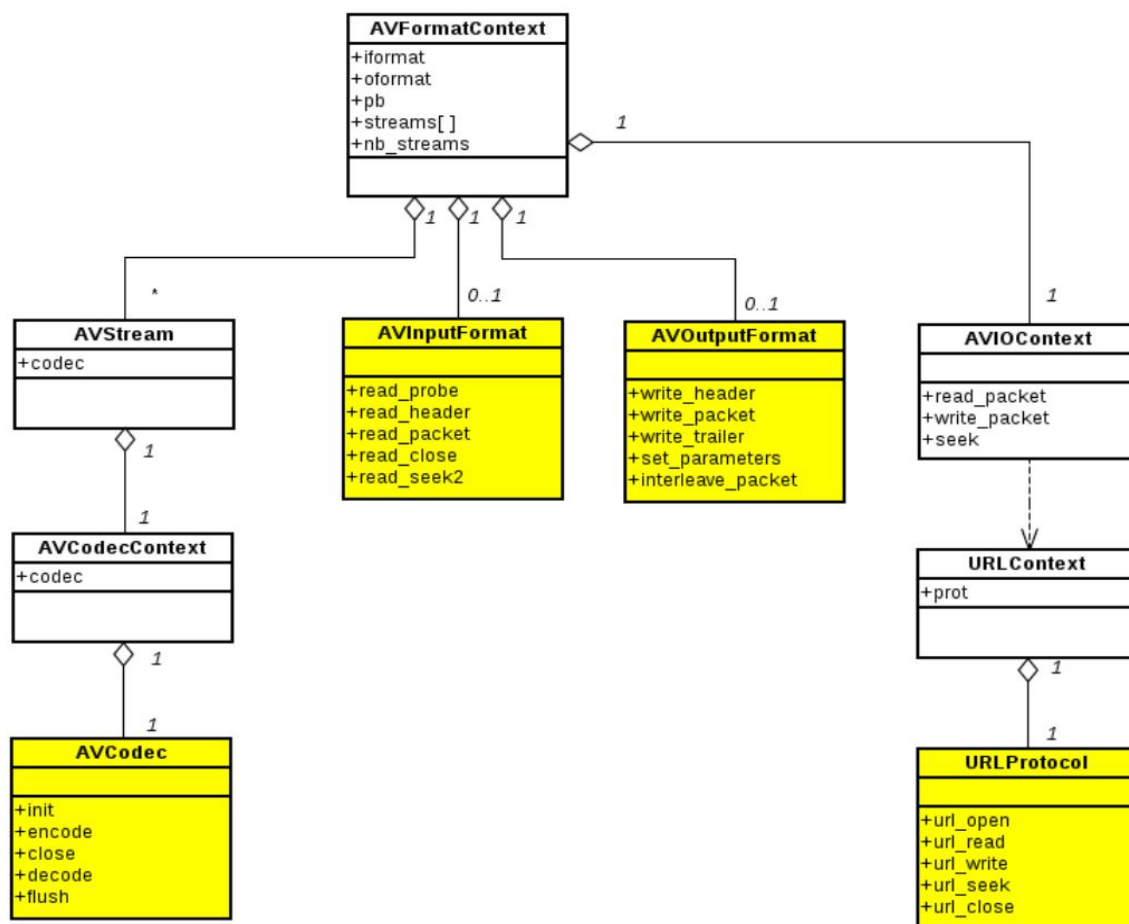


图 4-5 音视频解码数据结构图

(11) 网页图片播放器

图片播放器主要采用 JavaScript 脚本、Jquery 框架, 编写了一套面向对象封装的 PictureFrameJS 工具类, 来实现图片网页播放组件。该图片播放器支持这几个操作: 缩略图加载、图片放大缩小、拖拽、图片自动分组轮播等等酷炫的图片展示功能。

(12) 音视频播放器

音视频播放器主要采用了开源 MediaPlayer.swf 作为音视频的播放组件, 在网页中采用 <object> 标签对象的形式加载该插件。目前支持的视频格式众多。比如: mov、mpg、wmv、wav、mp4、flv、swf、avi、mp3 等, 在 Windows 操作系统的

IE 浏览器下视频播放器还可以用 `activex` 插件基于 `nsmp2inf.cab` 播放更多的音视频文件格式。

(13) 图片水印缩略图处理组件

主要采用 `JAVA` 开源工具包 `Thumbnailator` 和 `JAVA` 图形图像处理 `Image` 工具类编写了一套图片水印缩略图的处理组件。该组件还能根据设置水印通道透明度，通过压缩处理从而达到对大图片在网络间传输的提前预览效果。

4.2.2 系统流程设计

(1) 附件上传逻辑描述文件传输，以文档中文件的第一个文件的名称为文档的名称，若文档文件不存在，文件上传功能由个人中心中的文件上传模块统一实现。管理和支撑企业应用服务的附件存储和传输，保证文件存储的安全，以及实现附件操作的可追踪，存储校验逻辑。如图 4-6 所示。

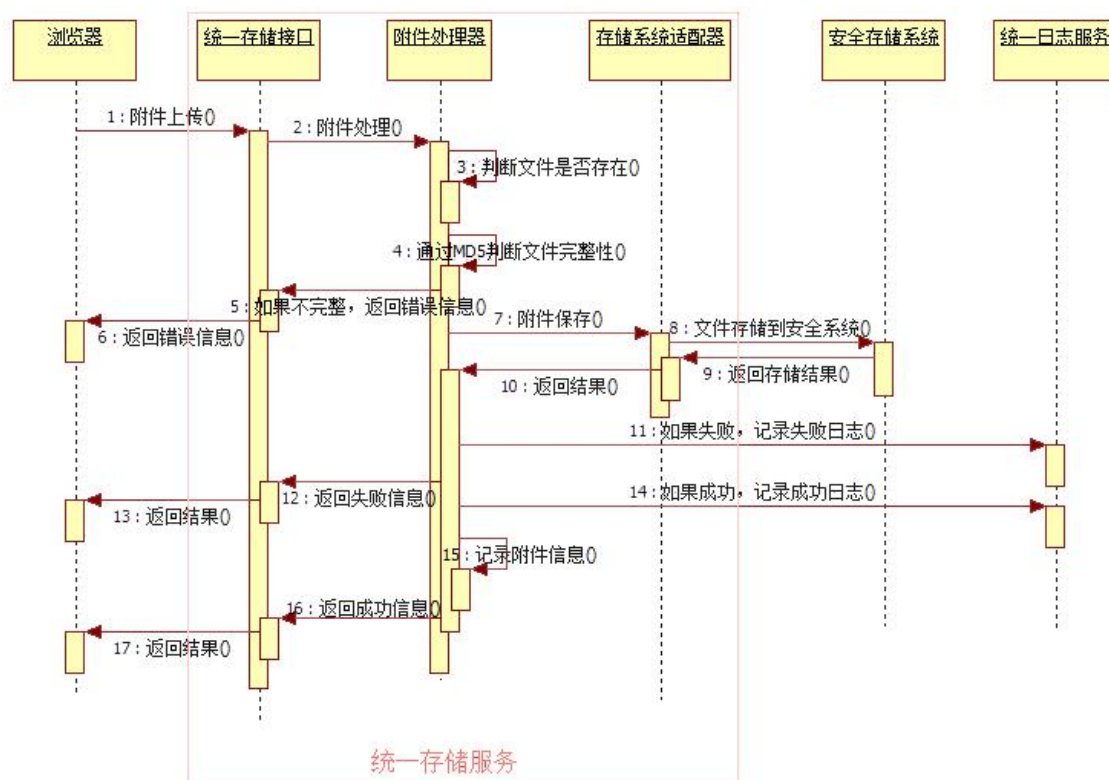


图 4-6 当文件不存在情况下的逻辑图

当文件已存在时可以对文档进行上传、发送、接收操作，并根据文档所确定的权限确定该用户是否具有转发，回复该文档的权限。如图 4-7 所示。

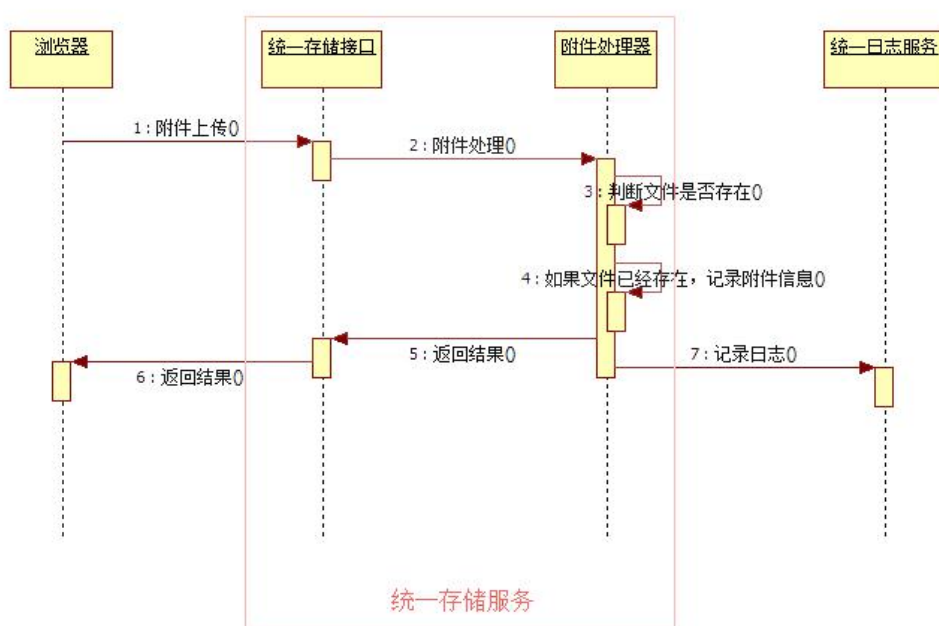


图 4-7 当文件已经存在情况下的逻辑图

(2) 更新文件信息逻辑文档保存的时候, 需要把文档和附件信息关联起来, 这时候需要调用该接口更新附件信息。如图 4-8 所示。

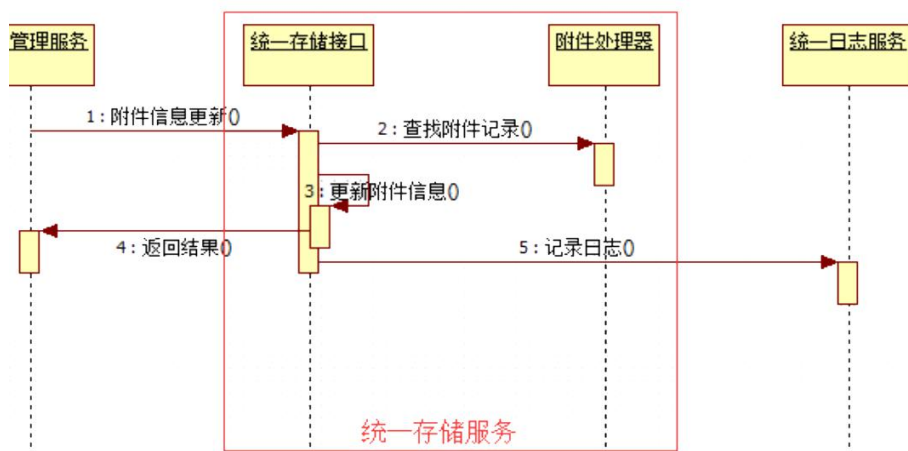


图 4-8 更新文件信息逻辑图

(3) 文件附件下载使用统一的下载组件, 在浏览器下载前, 对文件大小进行检测, 超过设定的阈值, 提示用户采用客户端方式操作, 但不强制。文件下载逻辑如图 4-9 所示。

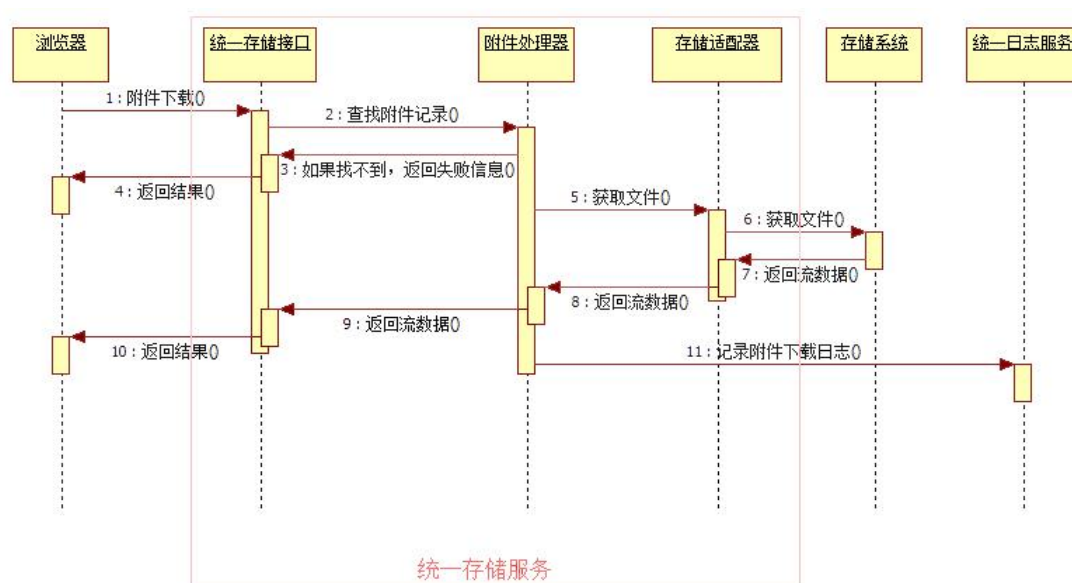


图 4-9 文件下载逻辑图

(4) 文件查询功能，用户可查询系统中所有的文件记录（用户有权限看的），按所属主文档、文件名称等进行查询，支持进行快速检索。用户可以查看系统中自己有权限看的所有文件，考虑到性能的问题，默认按时间段和所属系统进行过滤。对于用户自定义模式下的查询方式，通过构建个人中心去解决性能的问题。在个人中心中，以个人的维度保存“分类-附件”的相关数据，用户自定义模式下查询附件时，直接通过个人中心获取数据（不通过存储服务）。但是，在更新附件数据的时候，需要同时更新个人中心中的相关数据。作为一种补偿机制，个人中心可以定时请求存储中心的附件数据，重新构建个人中心中的“分类-附件”数据。文件查询逻辑如图 4-10 所示。



图 4-10 文件查询逻辑图

(5) 当删除文档附件后，需要把对应的附件记录 and 文件记录移到备份表中。如果文件没有被附件记录引用，那么需要把文件迁移到备份目录。调用统一日志服务，记录附件的删除信息。文件删除逻辑如下图 4-11 所示。

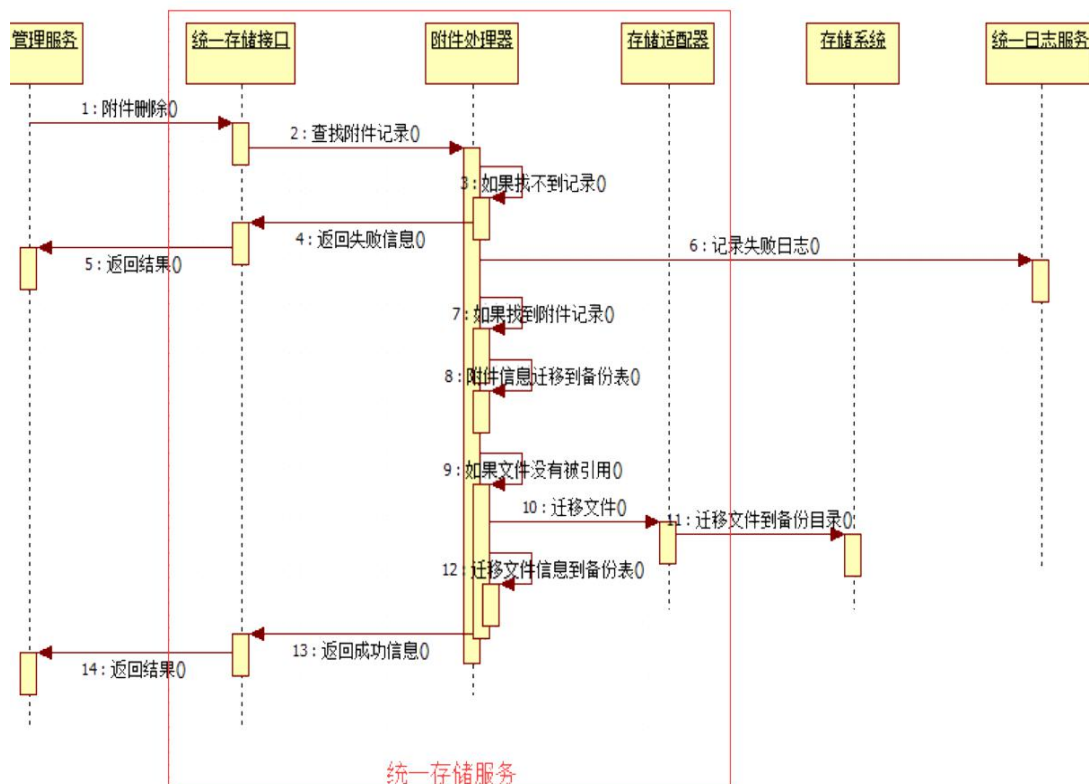


图 4-11 文件删除逻辑图

(6) 对于大附件的上传，考虑到传输速度以及稳定性的问题，前端先对文件进行切片；采用分片上传的方式，保存文件分片 MD5 和分片大小的值，文件上传的时候通过 MD5 码以及文件大小判断文件是否已存在，实现秒传功能。发送切片的时候可并发执行；统一存储服务收到切片后，把切片文件上传到安全存储系统，并在数据库中保存切片的相关信息；记录相应的日志。文件分片上传逻辑如图 4-12 所示。

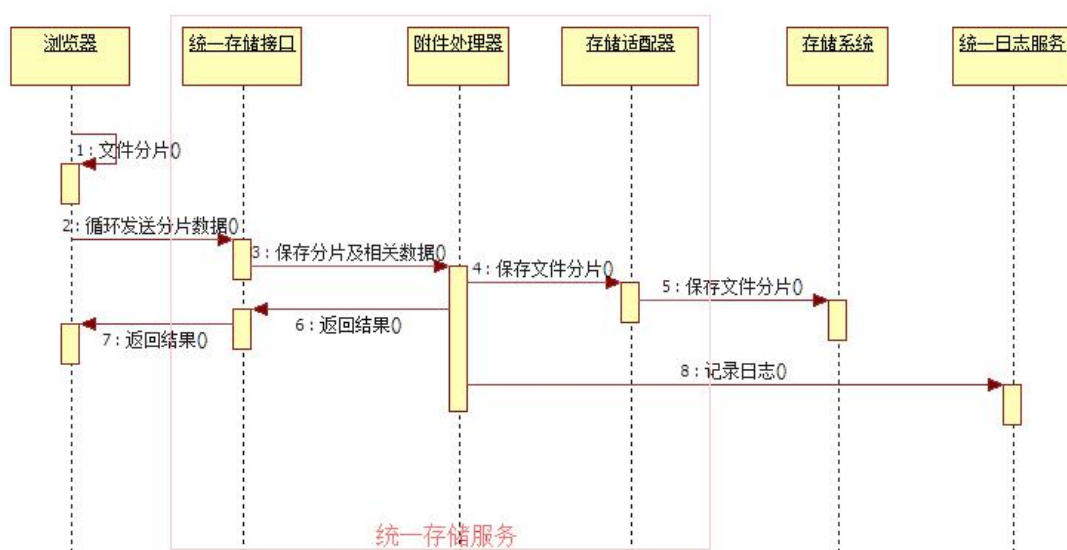


图 4-12 文件分片上传逻辑图

(7) 获取文件分片信息逻辑。根据文件大小计算分片数据信息；前端发起请求，获取每个分片的内容，可并发获取；前端获取到所有分片后，对分片进行合并。如图 4-13 所示。

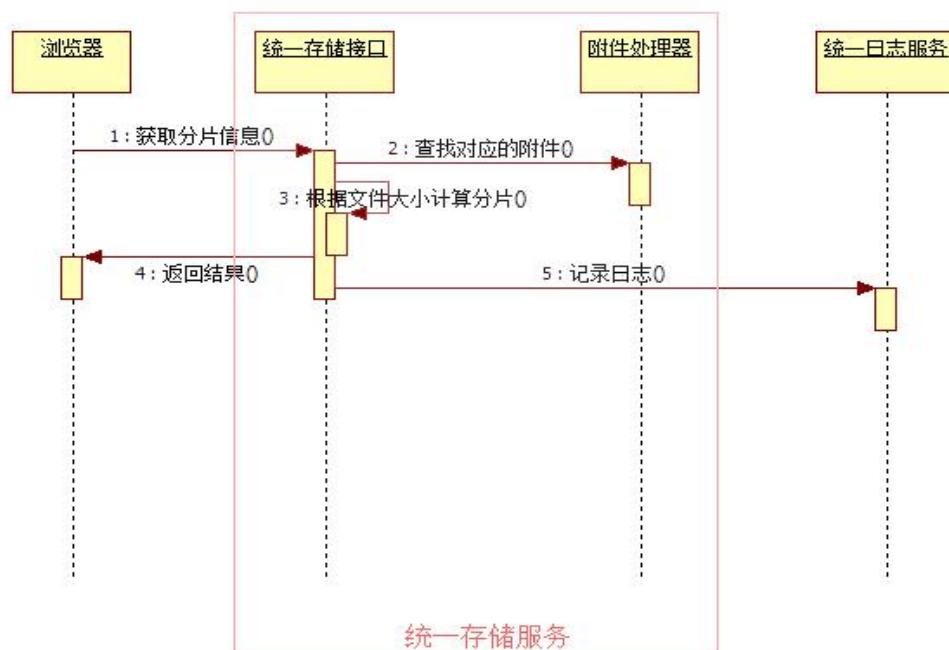


图 4-13 获取文件分片信息逻辑图

下载文件分片主要逻辑包括：分片传输每个切片文件，并带上 ID（用来区分是同一个文件的切片），切片的开始、结束位置，文件名、文件大小等信息；该附件展示名称、格式、大小，同时获取文件分片总数，开启多线程按进度下载，并

记录下载日志信息统计文件点击下载总次数。、如图 4-14 所示。

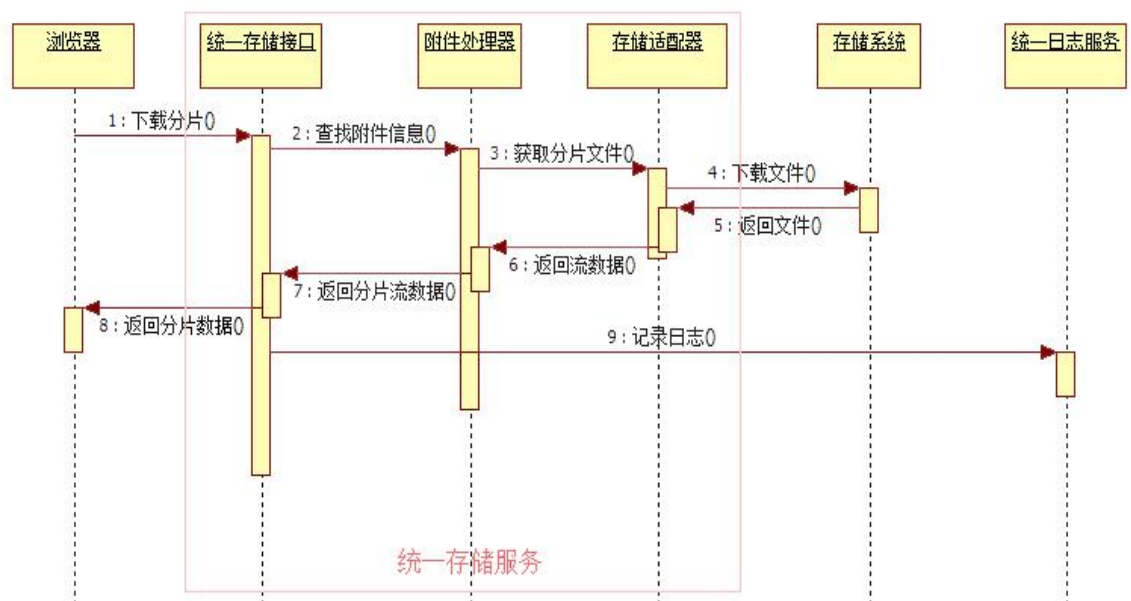


图 4-14 下载文件分片逻辑图

点击附件名称查看，附件应支持批量下载、批量收藏、单个预览，批量下载有系统自动生成新的压缩文件。批量下载附件，前端需要控制附件的大小（大附件只能单独下载）。后端对附件打包成 ZIP 后，传输到前端。如图 4-15 所示。

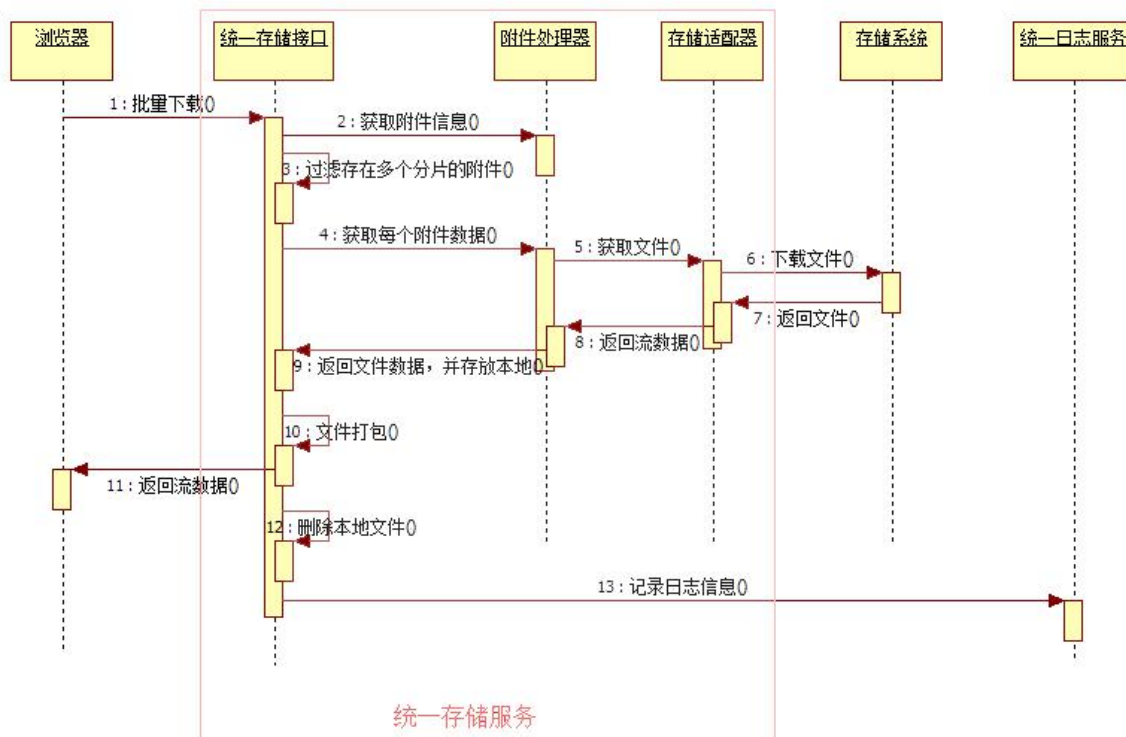


图 4-15 文件批量下载逻辑图

4.2.3 系统接口设计

(1) 附件上传接口。

文件附件上传使用统一的上传组件，点击新增弹出文件地址本勾选文件，勾选后附件展示。下拉选择相对应的密级，同时勾选该附件的状态（可下载、可复制、可截屏、可编辑、可阅读、可输出）。在文件上传后，若文件名不符合命名规则则进行命名。

接口名：/v1/sys/store/uploadAttachment（浏览器端访问）

/v1/sys/store/api/uploadAttachment（后端访问）

请求方式：POST

接口说明：上传附件，针对小附件

输入参数：

名称	类型	是否必填	说明
fileName	字符串	是	文件名称
attStream	流数据	是	

输出结果：

名称	类型	是否必填	说明
status	字符串	是	返回结果： 200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
attId	字符串	是	附件记录 ID

(2) 附件查询接口。

文件权限和文档权限分离，统一存储服务保存附件的权限信息，查询附件的时候要经过权限过滤。用户可查询系统中所有的文件记录（用户有权限看的），支持进行快速检索。

接口名： /v1/sys/store/api/queryAttachments（后端访问）

请求方式：GET

接口说明：查询附件

输入参数：

名称	类型	是否必填	说明
appName	字符串	否	系统来源
modelName	字符串	否	模块名称
modelId	字符串	否	模块 ID
key	字符串	否	
fileName	字符串	否	文件名称
attId	字符串	否	附件 ID

输出结果：

名称	类型	是否必填	说明
status	字符串	是	返回结果：200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
atts	JSON 数组	是	附件记录 ID
└appName	字符串	否	系统来源
└modelName	字符串	是	模块名称
└modelId	字符串	是	主文档 ID
└key	字符串	否	KEY
└creator	字符串	否	创建者
└createTime	字符串	是	创建时间
└fileName	字符串	是	文件名称
└fileSize	字符串	是	文件大小
└order	字符串	否	附件顺序
└right	JSON 对象	是	权限信息

(3) 附件下载接口。

根据附件 ID 下载对应的附件, 系统内的其它服务可以通过 RestFul 接口调用统一存储服务, 进行附件的下载操作。同时通过对应的文档入口, 进行附件下载的次数记录。

接口名: /v1/sys/store/downloadAttachment (浏览器端访问)

/v1/sys/store/api/downloadAttachment (后端访问)

请求方式: GET

接口说明: 下载附件, 针对小附件的下载

输入参数:

名称	类型	是否必填	说明
attId	字符串	是	附件 ID

输出结果:

名称	类型	是否必填	说明
status	字符串	是	返回结果：200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
attStream	流数据	是	

(4) 删除文档附件接口。

删除附件记录和对应的文件 (遵循“附件删除处理”逻辑)。当删除文档附件后, 需要把对应的附件记录和文件记录移到备份表中。如果文件没有被附件记录

引用，那么需要把文件迁移到备份目录。

接口名： /v1/sys/store/api/deleteAttachment（后端访问）

请求方式：GET

接口说明：删除附件，删除主文档的同时删除附件

输入参数：

名称	类型	是否必填	说明
appName	字符串	否	系统来源
modelName	字符串	是	模块名称
modelId	字符串	是	模块 ID
key	字符串	否	

输出结果：

名称	类型	是否必填	说明
status	字符串	是	返回结果：200 表示调用成功 其他均为失败
msg	字符串	否	错误信息

(5) 获取分片信息接口。

服务器端返回对应文件分片大小和总的片数信息，开启多线程模式实现大附件采用分片传输的方式进行。

接口名： /v1/sys/store/getAttachmentSlice（浏览器端访问）

/v1/sys/store/api/getAttachmentSlice（后端访问）

请求方式：GET

接口说明：获取附件分片信息，主要是针对大附件的下载

输入参数：

名称	类型	是否必填	说明
attId	字符串	是	附件 ID

输出结果：

名称	类型	是否必填	说明
status	字符串	是	返回结果：200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
fileName	字符串	是	文件名
fileSize	int	是	文件大小
sliceSize	int	是	分片大小
sliceCount	int	是	分片数

(6) 附件信息更新接口。

用于用户创建自定义目录或更新自定义文件存储目录，自定义文件夹创建、

更新、删除等操作。

接口名: /v1/sys/store/api/updateAttachmentMain (后端访问)

请求方式: POST

接口说明: 更新附件信息, 当文档保存的时候, 需要更新附件跟文档的关联关系以及附件权限信息。

输入参数:

名称	类型	是否必填	说明
attId	字符串	是	附件 ID
appName	字符串	否	系统来源
modelName	字符串	是	模块名称
modelId	字符串	是	模块 ID
key	字符串	否	
fileName	字符串	是	文件名称
creator	字符串	否	创建者
right	JSON 对象	是	附件权限信息, 格式如下: <pre>{ "reader":["123","456"], "editor":["123"], "download":[], "transfer":[] }</pre> "reader"表示对应的权限, 它的值为 JSON 数组存放有该权限的组织的 ID

输出结果:

名称	类型	是否必填	说明
status	字符串	是	返回结果: 200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
attId	字符串	是	附件记录 ID

(7) 大附件上传接口。

对于大附件上传, 考虑到传输速度以及稳定性的问题, 采用分片上传的方式。文件上传的时候通过 MD5 码以及文件大小判断文件是否已存在, 实现秒传功能。

接口名: /v1/sys/store/uploadAttachmentSlice (浏览器端访问)

/v1/sys/store/api/uploadAttachmentSlice (后端访问)

请求方式: POST

接口说明: 大附件上传, 按分片上传

输入参数:

名称	类型	是否必填	说明
appName	字符串	否	系统来源
modelName	字符串	是	模块名称
modelId	字符串	是	模块 ID
key	字符串	否	
fileName	字符串	是	文件名称
creator	字符串	否	创建者
fileId	字符串	是	文件 ID, 用来区分同一个文件的分片
attSliceStream	流数据	是	分片文件流
start	int	是	分片开始位置
end	int	是	分片结束位置
fileSize	int	是	文件大小
right	JSON 对象	是	附件权限信息, 格式如下: <pre>{ "reader":["123","456"], "editor":["123"], "download":[], "transfer":[] }</pre> "reader"表示对应的权限, 它的值为 JSON 数组存放有该权限的组织的 ID

输出结果:

名称	类型	是否必填	说明
status	字符串	是	返回结果: 200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
attStream	流数据	是	

(8) 附件批量下载接口。

实现批量下载所产生文件的密级规则逻辑, 附件应支持批量下载、批量收藏、单个预览, 批量下载有系统自动生成新的压缩文件, 且该文件的类型等级采用下载文件中文件的最高权限级别。

接口名: /v1/sys/store/downloadAttachmentBatch (浏览器端访问)

/v1/sys/store/api/downloadAttachmentBatch (后端访问)

请求方式: GET

接口说明: 批量下载附件, 前端需要控制附件的大小(大附件只能单独下载)。后端对附件打包成 ZIP 后, 传输到前端。

输入参数:

名称	类型	是否必填	说明
attIds	字符串	是	附件 ID, 以分号分隔

输出结果:

名称	类型	是否必填	说明
status	字符串	是	返回结果: 200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
stream	流数据	是	Zip 包流数据

(9) 大附件下载接口。

前端先发一个请求, 获取分片信息, 统一存储服务返回文件大小, 分片数等信息; 前端发起请求, 获取每个分片的内容, 可并发获取; 前端获取到所有分片后, 开启多线程下载进度模式最后对所有下载分片进行合并。

接口名: /v1/sys/store/downloadBigAttachment (浏览器端访问)

/v1/sys/store/api/downloadBigAttachment (后端访问)

请求方式: GET

接口说明: 下载附件分片, 需跟接口配合使用

输入参数:

名称	类型	是否必填	说明
attId	字符串	是	附件 ID
sliceNo	int	是	分片号, 从 1 开始

输出结果:

名称	类型	是否必填	说明
status	字符串	是	返回结果: 200 表示调用成功 其他均为失败
msg	字符串	否	错误信息
sliceStream	流数据	是	分片流数据

4.2.4 数据库设计

针对私有网盘应用系统的实现对其数据库进行分析与设计, 在实际的系统方案设计过程中通过 PowerDesigner 工具绘制数据库的逻辑模型, 进一步转换成下面的数据表结构关系图物理模型。数据库关系图如图 4-16 所示。

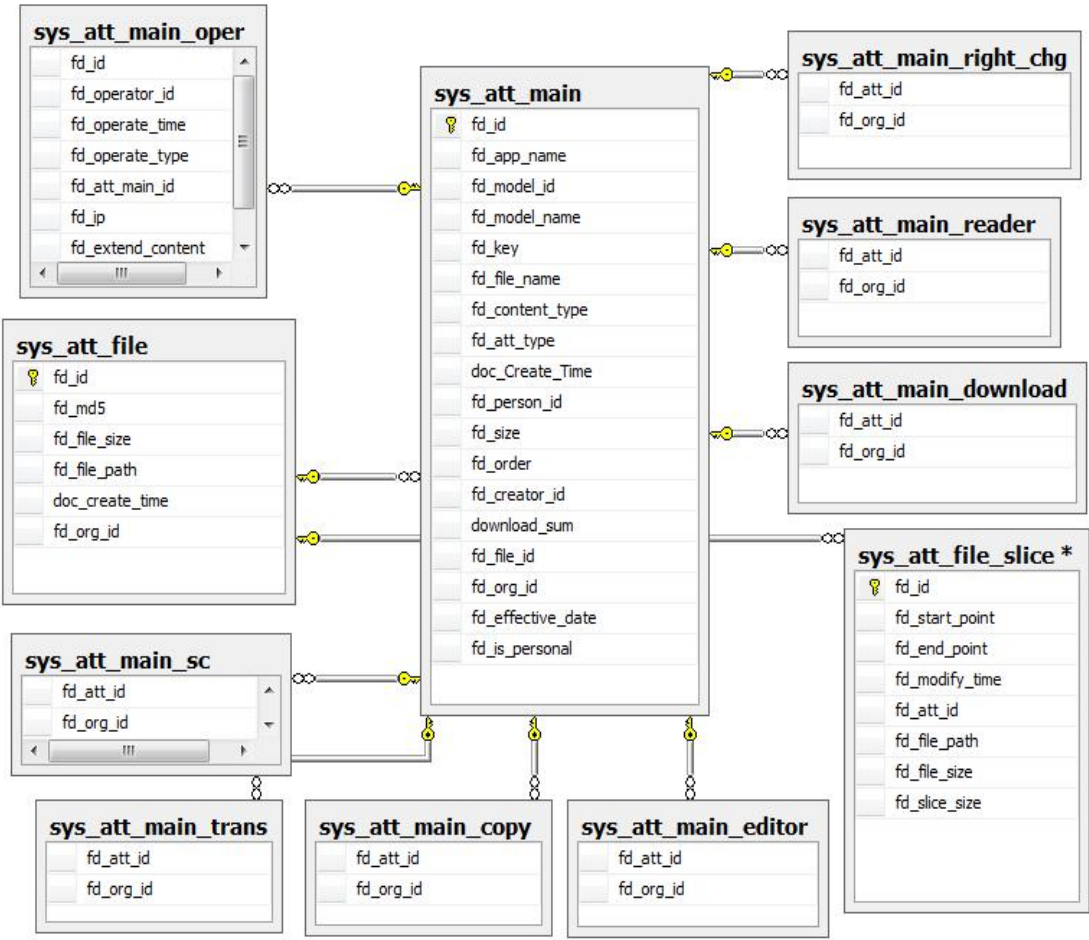


图 4-16 数据库表结构关系图

接下来的表格中，详细展示了上面数据库关系图中所涉及到的数据字典。

文件信息备份表模型，不同于附件信息主要是解决企业业务场景中，只是逻辑删除文件信息记录，不会直接删除文件真实实体，因此需要把对应的文件记录和移到备份表中。如果文件没有被附件记录引用，那么需要把文件迁移到备份目录中。

表 4-1 sys_att_file_bak 文件信息备份表

字段描述	字段	类型	说明
ID	fd_id	String	主键
文件大小	fd_file_size	Int	
文件路径	fd_file_path	String	非空
创建时间	doc_create_time	Date	非空
所属组织	fd_org_id	String	
扩展字段	Fd_extend	String	

附件存储信息表模型支撑统一存储服务主要是为了解决附件的统一存储、权限的控制以及跟安全存储系统的对接，系统内其它服务如果涉及到附件的上传、下载等操作，需要调用统一存储服务的接口，进行附件信息的存储和获取。

表 4-2 sys_att_main 存储附件信息表

字段描述	字段	类型	说明
ID	fd_id	String	主键
主文档 ID	fd_model_id	String	
所属模块	fd_model_name	String	
KEY	fd_key	String	
文件名	fd_file_name	String	非空
文件类型	fd_content_type	String	
附件类型	fd_att_type	String	
创建时间	doc_create_time	String	
文件大小	fd_size	int	非空
创建者	fd_creator_id	String	
附件顺序	fd_order	int	同一个文档下的附件的顺序
文件 ID	fd_file_id	String	非空，外键，关联 sys_att_file
所属组织	fd_org_id	String	
有效期	fd_effective_date	Date	
是否个人附件	fd_is_personal	boolean	1 表示个人文件,0 表示在文档中上传的文件
下载次数	download_sum	int	通过对应的文档入口，进行附件下载的次数
是否可以分发	fd_is_dispitch	boolean	
所属系统	fd_app	String	
关联附件	Fd_parent	String	表示该附件是由另一个附件转发（或者其它方式）而来
扩展字段	Fd_extend	String	

附件信息备份表模型主要是解决企业业务场景中，当删除文档附件后，需要把对应的附件记录和文件信息记录移到备份表中。如果文件没有被附件记录引用，那么需要把文件迁移到备份目录。

表 4-3 sys_att_main_bak 附件信息备份表

字段描述	字段	类型	说明
ID	fd_id	String	主键
主文档 ID	fd_model_id	String	
所属模块	fd_model_name	String	
KEY	fd_key	String	
文件名	fd_file_name	String	非空
文件类型	fd_content_type	String	
附件类型	fd_att_type	String	
创建时间	doc_create_time	String	
文件大小	fd_size	int	非空
创建者	fd_creator_id	String	
附件顺序	fd_order	int	同一个文档下的附件的顺序
文件 ID	fd_file_id	String	
所属组织	fd_org_id	String	
有效期	fd_effective_date	Date	
是否个人附件	fd_is_personal	boolean	1 表示个人文件, 0 表示在文档中上传的文件
下载次数	download_sum	int	通过对应的文档入口, 进行附件下载的次数
是否可以分发	fd_is_dispatch	boolean	
所属系统	fd_app	String	
关联附件	Fd_parent	String	表示该附件是由另一个附件转发(或者其它方式)而来
扩展字段	Fd_extend	String	

存储文件信息模型, 主要是在系统实际编码过程中, 采用面向对象的设计思想将附件信息与真实的文件实体分开进行存储管理, 降低代码组件之间的耦合度, 提高系统的通用性。

表 4-4 sys_att_file 存储文件信息表

字段描述	字段	类型	说明
ID	fd_id	String	主键
文件大小	fd_file_size	Int	
文件路径	fd_file_path	String	非空
创建时间	doc_create_time	Date	非空
所属组织	fd_org_id	String	
文件状态	Fd_secret_status	String	1:定密、2:预定密、3:销毁
扩展字段	Fd_extend	String	

附件分片信息表模型,借助 Hadoop 处理小文件块的思想,对于大附件的上传,考虑到传输速度以及稳定性的问题,采用前端先对文件进行切片;分片传输每个切片文件,并带上 ID (用来区分是同一个文件的切片),切片的开始、结束位置,文件名、文件大小等信息;发送切片的时候可并发执行;统一存储服务收到切片后,把切片文件上传到安全存储系统,并在数据库中保存切片的相关信息。

表 4-5 sys_att_file_slice 附件分片信息表

字段描述	字段	类型	说明
ID	fd_id	String	主键
分片开始位置	fd_start_point	int	
分片结束位置	fd_end_point	int	
修改时间	fd_modify_time	Date	
分片保存路径	Fd_file_path	String	
文件大小	Fd_file_size	Int	
文件名称	fd_file_name	String	
文件 ID	fd_file_id	String	
分片大小	Fd_slice_size	int	
分片 MD5	fd_md5	String	

4.3 系统编码实现

4.3.1 多文档上传下载组件实现

多文档上传下载组件主要实现在多线程状态下进行多个文件同时上传下载,并且具有文件拖拽上传与超大文件分片上传的功能,其具体核心代码如下:

```
/** JS 上传代码 */
```

```
$(document).ready(function() {
    changeUpload();
});
```

```
function changeUpload(){...}
```

```
/** JAVA 下载代码 */
```

```
public class FileDownload {
```

```
    /**
```

```
        * 文件下载
```

```
        * @param downFileFullName 待下载文件全路径
```

```
        * @param fileName 文件名称
```

```

* @param response HttpServletResponse 对象
*/
public static void downloadFile(String downFileFullName, String fileName,
    HttpServletResponse response) {
    /// 文件块的大小
    int BLOCKSIZE = 16 * 1024; // 每次拷贝文件块时重新指定文件块的大小
    (因此不能设为全局变量)
    ...
}

```



图 4-17 文件拖拽上传图

4.3.2 文档转换引擎实现

文档转换引擎采用的技术路线主要包括，微软 Office、金山 WPS、开源的 OpenOffice 软件转换协议，从而达到超高清文档 100% 转换成功的效果。其具体核心代码如下：

```

/** 微软 Office 转码工具类 */
public class MsOfficeConverter extends Common implements PDFConverter {
    // 微软 Office2010 及以上版本 Word 应用程序 PID
    public final static String WORDSERVER_STRING = "Word.Application";
    // 微软 Office2010 及以上版本 Excel 应用程序 PID
    public final static String EXECLSERVER_STRING = "Excel.Application";
    // 微软 Office2010 及以上版本 PPT 应用程序 PID
    public final static String PPTSERVER_STRING = "PowerPoint.Application";
    private static final int wdFormatPDF = 17; // Word 转 PDF 参数值
}

```

```

private static final int xlTypePDF = 0;//Excel 转 PDF 参数值
private static final int ppSaveAsPDF = 32;//PPT 转 PDF 参数值
public synchronized boolean doc2pdf(File docFile, File pdfFile) {...}
private boolean word2pdf(String docFilePath, String pdfFilePath){...}
private boolean excel2pdf(String docFilePath, String pdfFilePath){
    ActiveXComponent excel = null; //excel 运行程序对象
    Dispatch doc = null; //单个文档
    Dispatch documents = null; //所有文档
    // ComThread.InitSTA(); //初始化 COM 线程
    //经过实测，Microsoft Excel 2013 版转换 PDF 功能。
    ...
}
/**
 * 打开文件
 * @param docPath 要打开的文件，全路径
 * @return 打开的文件
 */
public Dispatch open(Dispatch documents, String docPath){
    return Dispatch.call(documents, "Open", docPath, false, true).toDispatch();
}

```



图 4-18 PPT 文档转换图

4.3.3 文档在线阅读器实现

文档在线阅读器是使用 Flex 技术开发的一款 Flash 播放插件与 JS 集成, 然后通过后台传递文件网络路径参数返回文件流信息最终显示在网页上, 并且该播放器还是实现了多页大文档的分页加载播放功能。其具体核心代码如下:

```
/** JS 代码 */
```

```
$(document).ready(function() {
    $('#documentViewer').mouseover(function(){
        EnableWheelScroll(false);//鼠标移动到阅读器部分, 禁用鼠标滚轴事件防止冒泡
    }).mouseout(function(){
        EnableWheelScroll(true);//鼠标离开阅读器部分, 用于滚动网页
    });
    getParentNav();
    initLayout();//设置文档阅读器布局
    $('#documentViewer').FlexPaperViewer({
        config : {
            DOC : encodeURIComponent(getDocumentUrl(startDocument)),
            Scale : 0.6,
        }
    });
});
```

```
/** JAVA 代码 */
```

```
/**
```

```
 * 对文件或文件夹进行压缩和解压
```

```
 * 注意: ZipOutputStream 类必须要 org.apache.tools.zip.ZipOutputStream
```

```
 */
```

```
public class ZipUtil {
```

```
//必须用 GBK 编码, 不然在加密压缩中文的时候会出现乱码
```

```
    public static String charset = "GBK";
```

```
    /**
```

```
     * 添加到压缩文件中
```

```
     */
```

```
    private void directoryZip(ZipOutputStream out, File f, String base) {...}
```

```

/**
 * 压缩文件
 */
private void fileZip(ZipOutputStream zos, File file) throws Exception {...}
/**
 * 解压缩文件
 */
private void fileUnZip(ZipInputStream zis, File file) throws Exception{ ...}
/**
 * 对文件 srcfile 进行先解密后解压缩,然后解压缩到目录 destfile 下
 * @param srcfile 要解密和解压缩的文件名
 */
public void decryptUnzip(String srcfile, String destfile, String keyStr){...}

```

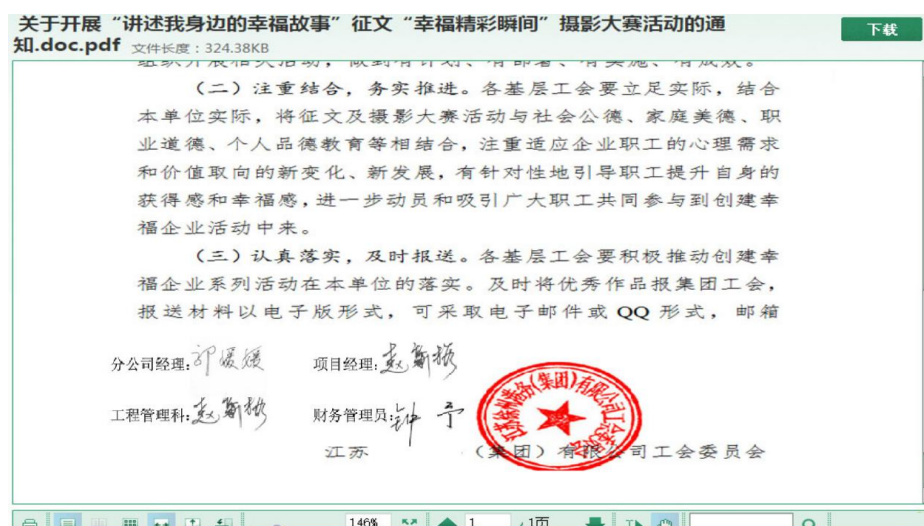


图 4-19 Word 签章签名文档在线阅读图

4.3.4 流媒体服务器实现

服务器需要提供视频地址列表中提供最新的 M 个视频片段地址。通过这种方式客户端可以通过轮询列表视频来同步客户端的视频直播。这样连接的效率可以大大提高。如下图 4-21 所示。

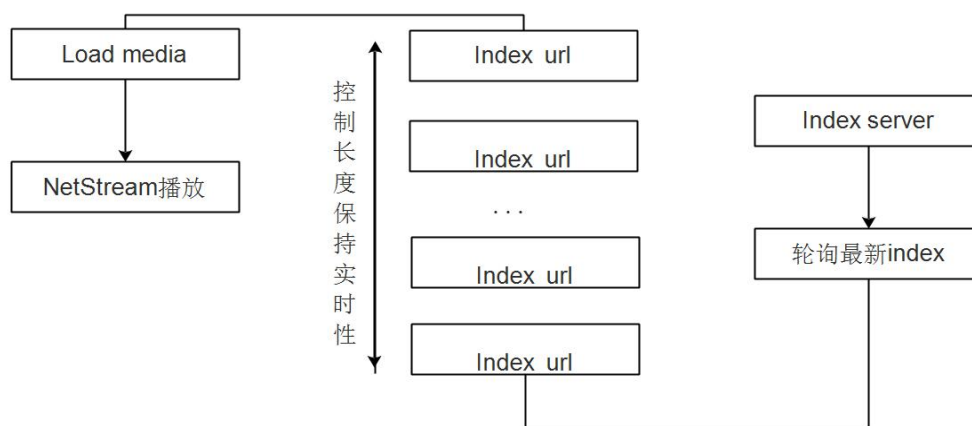


图 4-20 流媒体视频播放图

客户端维护切片列表队列，当队列长于某个阈值时队列将仅清空一个最近的地址，并允许队列继续增长直到下次检索地址为止。清除此队列的操作实际上是纠正了播放卡所造成的延迟，因此不应过多延迟视频直播。

关键核心代码如下：

(1) VideoEditor 的公共方法：

```

setonProgressListener(); //executeXXX 正在执行的百分比进度;
setEncoderBitrate(); //给 executeXXX 指定码率;
VideoEditor.isForceSoftWareEncoder=true; //强制 executeXXX 在编码时,采用软编码器;
cancel(); //取消正在执行的 executeXXX;
//当返回 null, 我们提供了 log 采集, 可通过 getErrorLog()得到错误信息;大部分是因为视频参数错误引起的.

```

(2) 写了 MediaInfo 辅助类, 用来很快的获取视频的基本信息, 以方便实际参数的参考, 使用如下:

```

MediaInfo info=new MediaInfo("要获取视频的路径");
if(info.prepare()){
...
//如果返回 true,得到视频宽度,高度,码率,帧率,时长,编码器,总帧数,是否有 B
帧,旋转角度, 音频采样率,音频通道数, 音频码率,是否有音频,视频,视频是否旋转
等参数;
}

```

(3) 文件创建,删除类 `LanSongFileUtil.java` 方便你在编辑时的各种文件创建、删除,判断等操作,也可以先用 `MediaInfo` 判断当前视频的各种信息。

(4) 直接导入 `lansongsdk` 这个 module 后;在工程的 `setting.gradle` 中增加: `,'LanSongSDK'`;并在 `app.gradle` 中增加: `implementation project(':LanSongSDK')`在代码开始的时候,增加 SDK 初始化代码:

```
LanSoEditor.initSDK(getApplicationContext());
```

(5) 流媒体服务器后端代码 80% 的问题是因为文件传输错误,可用 `MediaInfo.checkFile("文件绝对路径")`;来查看打印信息,根据打印信息的提示排除问题从而来查看打印信息,。如返回 `xxx pixel -1` 之类,说明找不到对应的节码器;如其他信息,查看对应的关键字,大部分是因为命令错误,或没有覆盖之前的文件等等。是否强制使用硬件编码器;默认先硬件编码,如果无法完成则切换为软编码解析参数失败返回 1,无输出文件 2;输入文件为空 3, sdk 未授权-1;解码器错误 69,收到线程的中断信号 255,如硬件编码器错误,则返回 26625---26630。

4.3.5 音视频转码引擎实现

(1) 在代码转换中,描述编解码器上下文的数据结构使用 `AVCodecContext` 的主要核心类库,包含许多编解码器所需的参数信息。音频倒序,和视频倒序类似,把原来正常的声音,处理成从后向前的声音。那么要注意的是,此处理会占用大量的内存,建议视频最好是 480x480 的分辨率,并且不要过长,尽量在 15 秒内。源文件完整路径,原文件可以是 mp4 的视频,也可以是 mp3 或 m4a 的音频文件;视频倒序:比如正常的视频画面是一个人从左边走到右边,倒序后,人从右边倒退到左边,即视频画面发生了倒序。如您的视频过大,则可能导致:Failed to inject frame into filter network: Out of memory;这个是正常的,因为已超过 APP 可使用的范围,内存不足。

音频转码核心代码:

```
/** 最简单的调用方法: AudioEditor audiotor=new AudioEditor();
 * auditor.executeXXXXX();//阻塞执行,要放到 AsyncTask 或 Thread 中执行*/
public class AudioEditor {
    /** 使用软件编码的列表;*/
    public static String[] useSoftEncoderlist = {};
    /** 获取用户使用 sdk 的版本类型 @return */
    public static native int getLanSongSDKType();
    /** 精确裁剪 @param videoFile @param startS @return */
```

```

public String executeCutVideoExact(String videoFile, float startS);
/** 来自于网络, 没有全部测试. 获取视频的缩略图
 * 提供了一个统一的接口用于从一个输入媒体文件中取得帧和元数据。
 * @param path 视频的路径 @param width 缩略图的宽
 * @param height 缩略图的高 @return 缩略图 */
public static Bitmap createVideoThumbnail(String path, int width);
/** 把 mp3 转为 aac 格式, 需要编解码, 因为是软解后软编码, 会需要一些时
间.
 * @param mp3Path @return 目标文件路径, 后缀 m4a */
public String executeConvertMp3ToAAC(String mp3Path);
/** 把分段录制的视频, 拼接在一起; 注意:此方法仅仅使用在分段录制的场
合
 * 注意:此方法仅仅使用在分段录制的场合
 * @param mp4Array */
public String executeConcatMP4(String[] mp4Array) {
    //第一步,先把所有的 mp4 转换为 ts 流
//第二步: 把 ts 流拼接成 mp4
//第三步:删除临时生成的 ts 文件.
}
/** 不同来源的 mp4 文件进行拼接; 拼接的所有视频分辨率必须一致;
 * @param videos 所有的视频
 * @param ignorecheck 是否要忽略检测每个每个视频的分辨率,
 * 如果您确信已经相等,则设为 false;
 * @return 输出视频的路 */
public String executeConcatDiffentMp4(ArrayList<String> videos);
}

```

(2) 设置多媒体文件中的视频元数据的角度, 一个多媒体文件中有很多种元数据, 包括音频轨道、视频轨道、各种元数据、字幕其他文字等信息。这里仅仅更改元数据中的视频播放角度, 当视频播放器播放该视频时, 会得到"要旋转多少度"播放的信息, 这样在播放时就会旋转后再播放画面。此设置不改变音视频的各种参数, 仅仅是告诉播放器,"要旋转多少度"来播放而已。适用在拍摄的视频有 90 度和 270 度的情况, 想更改这个角度参数的场合。原视频需要更改的角度, 给视频旋转角度, 这里只是旋转画面的角度, 而不会调整视频的宽高。需要旋转角度

的原视频，把视频填充成指定大小的画面，比视频的宽高大的部分用黑色来填充。源视频路径，填充成的目标宽度参数需要是 16 的倍数。填充成的目标高度，参数需要是 16 的倍数把视频画面放到填充区时的开始 X 坐标，把视频画面放到填充区时的开始 Y 坐标。

视频转码核心代码：

```
/** 最简单的调用方法: VideoEditor veditor=new VideoEditor();
 * veditor.executeXXXXX();//阻塞执行，要放到 AsyncTask 或 Thread 中执行*/
public class VideoEditor {
    /** 视频水平镜像，即把视频左半部分镜像显示在右半部分
     * 【此方法用到编解码】 @param srcPath 源视频路径 @return */
    public String executeVideoMirrorH(String srcPath);
    /** 视频垂直镜像，即把视频上半部分镜像显示在下半部分
     * @param srcPath 源视频路径 @return */
    public String executeVideoMirrorV(String srcPath);
    /** 视频垂直方向反转 @param srcPath 原视频 @return */
    public String executeVideoRotateVertically(String srcPath);
    /** 视频水平方向反转 @param srcPath 原视频 @return */
    public String executeVideoRotateHorizontally(String srcPath);
    /** 增加编码器,并开始执行;
     * @param cmdList @param bitrate @param dstPath
     * @param isHWEnc 是否使用硬件编码器; 如果强制了,则以强制为准;
     @return */
    public int executeWithEncoder(List<String> cmdList,int bitrate);
    /** 检测是否需要软编码; @return */
    public boolean checkSoftEncoder();
    /** 把数据变成 16 的倍数, 以大于等于 16 倍数的为准; 比如:
     * 16-->返回 16; 17---31-->返回 32; @param value @return */
    public static int make16Next(int value);
}
```

(3) 把多张图片转换为视频。需要注意，这里的多张图片必须在同一个文件夹下，并且命名需要有规律，比如名字是 r5r_001.jpeg r5r_002.jpeg, r5r_003.jpeg 等多张图片，需要统一的分辨率，如分辨率不同，则以第一张图片的分辨率为准，后面的分辨率自动缩放到第一张图片的分辨率的大小、保存图片的文件夹、图片

的文件名有规律的前缀、每秒钟需要显示几张图片等。

视频缩略图抓取核心代码:

```
/** 最简单的调用方法: PicVideoCrawtor picCrawtor=new PicVideoCrawtor();
 * picCrawtor.executePicVideoCrawtor();//抓取视频图片执行*/
public class PicVideoCrawtor {
    /** 使用软件编码的列表; */
    public static String[] useSoftEncoderlist = {};
    /** 是否强制使用软件编码器,全局变量
     * 默认先硬件编码,如果无法完成则切换为软编码*/
    public static boolean isForceSoftWareEncoder=false;
    /** 获取用户使用 sdk 的版本类型 @return */
    public static native int getLanSongSDKType();
    /** 精确裁剪 @param videoFile @param startS @return */
    public String executeCutVideoExact(String videoFile, float startS);
    /** 把 mp3 转为 aac 格式, 需要编解码, 因为是软解后软编码, 会需要一些时
    间.
     * @param mp3Path @return 目标文件路径, 后缀 m4a */
    public String executeConvertMp3ToAAC(String mp3Path);
}
```

com.videoEditor.NoFree 文件夹中的类的所有功能包括: 底层硬件加速编码器、极速视频压缩功能、视频分段录制、支持回删、支持美颜、聚焦、亮度调节, 按下录制拍摄图片; 编辑功能界面包括: 涂鸦、增加文字、增加图片、变速、时长裁剪、画面裁剪、定制的功能永久稳定使用。速度优化.很复杂的效果,也只需几行代码就可以实现,可以让二次开发人员少走很多弯路;当数据不是 16 的倍数的时候,把他调整成 16 的倍数,以最近的 16 倍数为准。举例如下:

16, 17, 18, 19,20,21,22,23 ==>16;

24,25,26,27,28,29,30,31,32==>32;

如果是 18,19 这样接近 16,则等于 16, 等于缩小了原有的画面,如果是 25,28 这样接近 32,则等于 32, 等于稍微拉伸了原来的画面,因为最多缩小或拉伸 8 个像素,还不至于画面严重变形,而又兼容编码器的要求,故可以这样做。获取 lansosdk 的建议码率;这个码率不是唯一的,仅仅是建议这样设置。如果对码率理解很清楚或有一定的压缩要求,则完全可以不用我们的建议自行设置。流媒体视频转换如下图 4-23 所示。

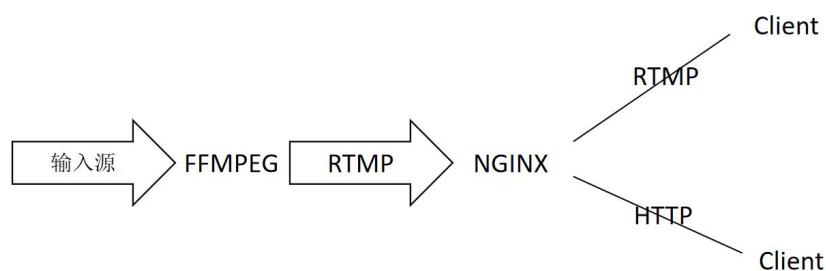


图 4-21 流媒体视频的转换图

4.3.6 图片播放器实现

图片播放器类似于 QQ 空间的照片浏览播放器功能，可以将多张缩略图同时浏览和原图可缩放大小、拖拽位置，图片自动播放功能。其具体核心代码如下：

/** JS 代码 */

```

PictureFrame.prototype = {
    // 参数
    arrowImageNum    : 6,           //初始话缩略图显示数量
    colorToLi        : '#bd9a7f',   //设置缩略图背景色
    isZooming        : false,       // 是否正在缩放图片
    isDraging        : false,       // 是否正在拖拽图片
    orderArray       : [], //缩略图显示顺序
    defaultImgUrl    : 'pictureView/images/loading_big.gif', //缩略图默认地址
    activeImage      : null,        // 图片加载容器
    lng              : {
        'prev'       : '上一张',
        'next'       : '下一张',
        'autoplay'   : '自动播放',
        'downplay'   : '停止播放',
        'loading'    : '图片加载中...',
        'loaderror'  : '图片加载错误'
    }, // 提示语言

    // 初始化
    initialize : function() { ...},
    // 设置框架布局
    setFrameLayout : function() { ...},

```

```
//图片加载错误事件
loadError : function() { ...},
//阻止冒泡时间
stopEvent: function(evt){ ...}
}
/** JAVA 代码 */
public class MinPictureUtil {
    public static BufferedImage logoBuffer = null;
    /**
     * 创建缩略图
     * @param file 上传的文件流
     * @param height 最小的尺寸
     */
    public static void createMiniPic(File file, float width, float height){
        createSizePic(file.getAbsolutePath(), (int)width, (int)height);
        ...
    }
}
```



图 4-22 资源管理器--多文件类型文件图

4.3.7 音视频播放器实现

企业私有云盘为自定义开发了 jar 包源码，利用 java 对 FFMPEG 进行二次封

装，能将直播视频输入流转换成 HLS 切片，并且支持批量自动创建文件夹、自动重连、音视频混流、转码等功能。（使用本系统的音视频播放器前，请先安装 1.6 以上版本的 java 运行环境(JRE)，并配置 java 环境变量）打开 HLSServer 文件夹，并用记事本等编辑工具打开 config.xml 文件里面的播放器自定义标签解释如下：

<config>标签：跟标签，标签内的 type 属性统一写为 HLS（在开发的 rtmp 推流工具中此属性将为 RTMP）。

<CommBGMusic>标签：全局背景音乐文件名，必填。请将音乐文件放置 MusicResource 文件中，并将文件名写到该标签下。

<ServerPath>标签：本地流媒体服务器 HLS 文件夹路径（注意末尾需加上“\”），必填。当目录不存在时，工具将会自动创建目录；在每次启动时，工具将自动把该目录清空。

<CameraList>标签：摄像头（视频源）列表标签，标签内可有多于一个<Camera>标签。

<Camera>标签：摄像头（视频源）标签。 <name>标签：m3u8 文件文件名（不带文件类型后缀），必填。

<URL>标签：视频源地址，注意 xml 中规避的字符必须使用转义字符，如“&”用“&”代替。

<OutputVideoFormat>标签：强制输出视频编码格式，不填则使用源视频编码。

<OutputAudioFormat>标签：强制输出音频编码格式，不填则使用源音频编码。

<dirName>标签：子 HLS 文件夹，在工具运行的过程中将自动生成配置的文件夹，不填则默认值和 name 相同。

<BgMusic>标签：背景音乐文件名，请将音乐本件拷贝到 MusicResource 目录下。

<LogTag>标签：日志标签标签。

播放器核心代码：

```
<html>
<head>
<title>音视频播放器页面</title>
<style type="text/css">
html, body{margin: 0px; padding: 0px;}
</style>
</head>
<body>
<c:choose>
<c:when test="{fn:escapeXml(requestScope.extName)} == 'avi'"
```

```

    || fn:escapeXml(requestScope.extName) == 'mov'
    || fn:escapeXml(requestScope.extName) == 'mpg'
    || fn:escapeXml(requestScope.extName) == 'wmv'
    || fn:escapeXml(requestScope.extName) == 'wav'
    || fn:escapeXml(requestScope.extName) == 'mp3'}">
<object id="SPXX_player" style="width: 100%; height: 100%;"
    classid="clsid:22d6f312-b0f6-11d0-94ab-0080c74c7e95"
    codeBase="http://activex.microsoft.com/activex/con
n/nsmvp2inf.cab#version=6,4,5,715"
    type="application/x-oleobject" altHtml=""
    standby="loading microsoft windows media player components...">
</object>
</c:when>
</c:choose>
</body>
</html>

```



图 4-23 WMV 视频播放图

4.4 系统集成环境

集中配置中心主要是为企业私有网盘系统内的所有应用服务提供统一的配置管理，信息包含如应用服务数据字典信息、应用服务的权限、安全存储配置等基本配置信息。运行于 JVM 环境中，采用 SpringCloud 技术、Web 容器采用 Webspere，支持分布式高可用部署。如图 4-26 所示。

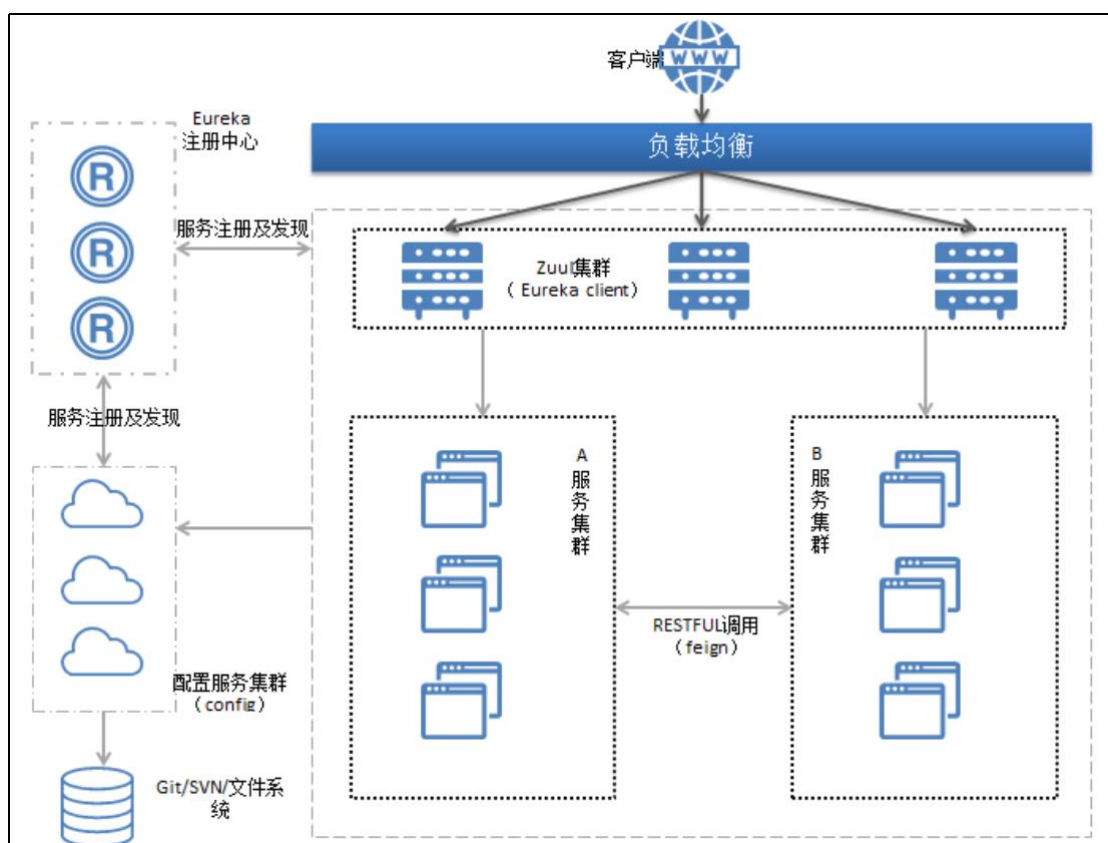


图 4-24 系统集成配置环境

集中配置中心负责网盘系统应用服务的配置管理，如果应用服务需要做配置信息统一管理，应用服务本身也就是配置中心客户端。当应用服务启动时，会同步配置中心的配置信息（只同步本服务的配置信息）。集中配置中心本身也是eureka客户端，需要将本身服务注册到eureka服务端；集中配置中心的配置信息读取，应用服务在开发时需要对应的访问规则，进行配置；集中配置中心服务端和客户端的信息同步，主要是保障配置信息的可用性；集中配置中心提供有配置变更，无需停机的配置刷新功能。

集中配置中心主要用于解决系统内服务的配置信息管理，为保证配置中心本身的高可用、安全性以及应用服务读取配置的可靠性，决策如下：

- (1) 系统内服务，如果涉及在开发设计层面的固定配置信息，均需要交由集中配置中心统一管理。
- (2) 集中配置中心的访问需要做安全控制，且服务在系统外不允许访问（通过服务网关配置解决）。
- (3) 集中配置中心服务端和客户端本身会有服务信息同步，一旦他们之间的通讯有问题，会导致配置生效时间间隔问题，需要保证配置中心的高可用性，应部

署多台。

4.5 本章小结

本章内容基于需求分析，完成了软件系统的总体架构设计、系统功能设计，系统详细设计。在系统设计的基础上，解释了软件系统的一些主要功能。系统架构采用了基于 B/S 结构的分层模型，实现了系统的数据、业务逻辑和用户界面的分离，提高了系统的开发效率和运行效率。

第五章 系统测试

系统在开发过程中难免会出现 Bug，通过软件测试来排除软件中出现的隐形错误从而来提高软件的开发效率，衡量软件实现过程中是否与用户需求相契合。本章从系统测试目标、搭建系统环境，对软件功能和性能进行测试，然后进行测试分析，最后给出相应测试结论。

5.1 系统测试

5.1.1 测试目标

软件开发流程的重要部分之一就是软件测试。还有一种新的软件开发模型为测试驱动软件开发。不仅有助于软件开发过程的管理而且还能把握软件的质量。系统开发与测试的关系如图 5-1 所示。

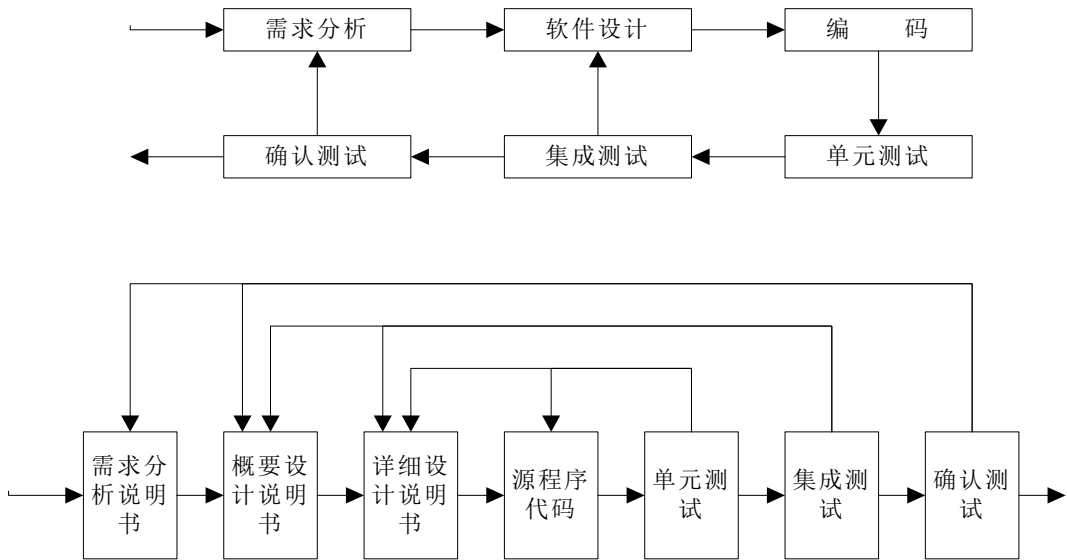


图 5-1 开发与测试关系示意图

系统测试的目的在于解决程序的正确性，尽量消除程序中的各种显性或者隐性的差错。测试的对象不仅仅包括软件的程序本身，也包括软件设计过程中的需求获取与分析阶段、系统总体设计和详细阶段的各项成果性的说明文档。根据测试的原则，对本系统的测试目标定位如下。

(1) 进行软件系统的功能测试。文件上传下载功能、文件视图资源管理器功能、文档在线预览功能、图片预览播放功能、音视频动画播放功能、在线解压缩

功能等等。

(2) 进行软件安全性测试，通过对系统具体页面和功能的安全测试，验证系统是否具有非授权访问、数据泄露等问题，以及可预见的安全性问题的处理策略是否得当。

(3) 进行软件可靠性测试。验证一定数量用户并发访问时是否存在系统可靠性故障以及在连续运行过程中是否存在可靠性问题。测试流程如图 5-2 所示：

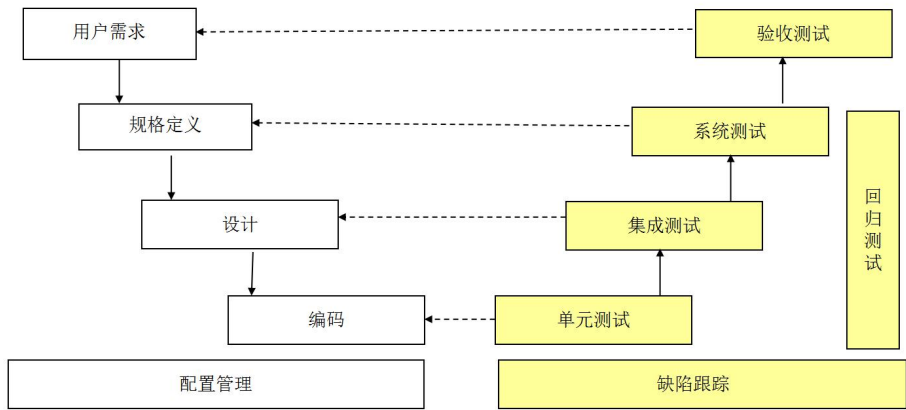


图 5-2 软件开发对应的测试过程图

5.1.2 测试环境

(1) 硬件环境配置（服务器端配置和客户端配置）

表 5-1 服务器端配置

服务器名	操作系统	CPU	内存	磁盘	软件环境
文件转换服务器（1 台）	Redhat 6.5 以上	2 颗 10 核心（2.8GHz 以上）	128GB	600GB15k 外接存储按需扩展	IHS
流媒体服务器（1 台）	Redhat 6.5 以上	2 颗 10 核心（2.8GHz 以上）	256GB	600GB15k 外接存储按需扩展	JDK1.7 和 JDK 1.8（64bit） WAS 8.5.5
数据库服务器（1 台）	Win2008R 以上	2 颗 10 核心（2.4GHz 以上）	256GB	600GB15k 外接存储按需扩展	MySql 数据库
安全 NAS 存储	支持集群部署，挂接后端存储				
集中存储系统	支持 FC/NAS 等协议和接口				

表 5-2 客户端配置

设备名	操作系统	CPU	内存	磁盘	软件环境
客户端	Win XP (32/64)专业版 Win 7(32/64)专业版 Win 10(32/64)专业版	1.8GHz 以 上	2GB 以上	100GB 以上	Office2003 以上 IE8 浏览器 360 安全浏览器

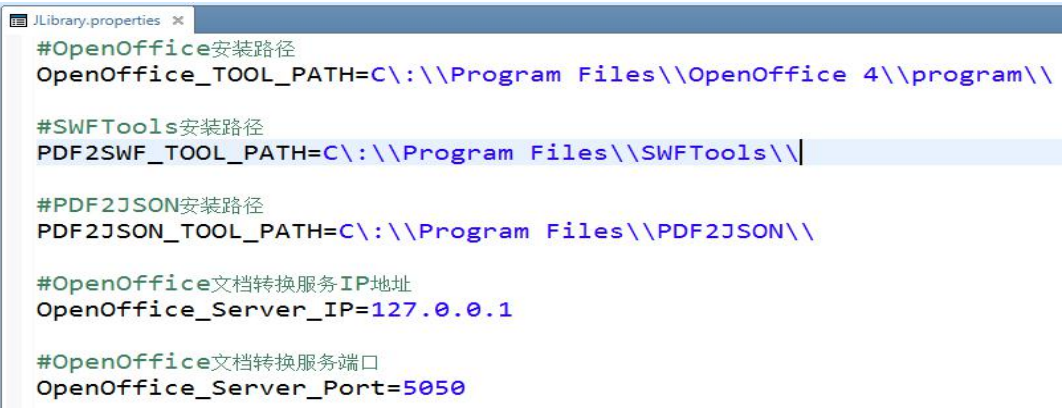
(2) 软件环境配置

使用企业私有网盘系统之前需要安装第三方软件有微软 Office2010 及以上版本、金山 WPS2015 及以上版本、开源 OpenOffice 4.0.1 及以上版本、SWFTTools -2013-04-09-1007 及以上版本，并安装到默认安装目录下，不要随意改动安装目录，否则会导致文档转码服务异常。如果确实需要更改安装目录，那么要同时更改企业私有网盘系统配置文件指定第三方工具的安装路径。

安装 JDK 和 Tomcat 形成之前，目前需要手动安装和配置 JAVA 程序运行的一些环境变量。Tomcat 安装时要记住服务器的安装目录和端口号。

(3) 网盘 APP 部署

直接将 APP 程序文件放置到安装好的 Tomcat 服务器的 webapps 目录下，修改配置文件，重启服务器即可。如图 5-3 所示。



```
JLibrary.properties
#OpenOffice安装路径
OpenOffice_TOOL_PATH=C:\\Program Files\\OpenOffice 4\\program\\

#SWFTTools安装路径
PDF2SWF_TOOL_PATH=C:\\Program Files\\SWFTTools\\

#PDF2JSON安装路径
PDF2JSON_TOOL_PATH=C:\\Program Files\\PDF2JSON\\

#OpenOffice文档转换服务IP地址
OpenOffice_Server_IP=127.0.0.1

#OpenOffice文档转换服务端口
OpenOffice_Server_Port=5050
```

图 5-3 网盘系统配置管理

特别注意：端口 5050 是企业私有网盘程序专门用来与 OpenOffice 连接文档转换服务的，不能被占用，也不能将 Tomcat 等 web 应用服务器配置成该端口，一旦占用请在配置文件更改。

(4) 访问测试入口

当我们安装部署好企业私有网盘系统后就可以进行相应的使用测试工作了。
首页访问地址：<http://企业私有网盘服务器 IP:端口/JLibrary>
说明：（企业私有网盘服务器 IP:PORT 企业私有网盘 APP 程序文件部署的服务器 IP 地址和端口号，如：127.0.0.1:8080）

5.1.3 功能测试

对该私有网盘系统最重要的模块功能进行测试。软件测试用例的编写通常包括以下内容：

1，测试用例编号；2，测试项目；3，测试标题；4，重要级别；5，预置条件/前提条件；6，操作步骤；7，预期结果/期望结果；8，测试结果/实际结果；9，复现概率等等。重要级别也可称为优先级，是对缺陷的严重程度做出的判断。一般分为高（**highs**）级别、中（**mediums**）级别，低（**lows**）级别等。在一些软件中也分为严重级别、重要级别、轻微级别等，只是名称的不同而已。在实际工作中，我们还要根据项目内容及 **bug** 详情来做具体的判断。另外还有一种小版本确认测试，也称为“冒烟测试”，其重要级别要优于高级别，但并不是所有项目必须。本企业私有网盘具体测试用例及结果见表 5-3 所示。

表 5-3 测试用例

序号	测试功能	测试操作	预期结果	实际测试结果
1	文件上传	上传各种类型的文件	提示上传成功，转入列表页面	达到预期结果
2	文件下载	下载各种类型的文件	点击下载按钮，进入下载页面	达到预期结果
3	文件删除	删除已有文件	点击删除按钮，完成删除操作	达到预期结果
4	文件查找	用户查找文件	进入查找界面，完成文件查找	达到预期结果
5	添加水印	在文档中添加文字图片水印	能够正常添加显示水印	达到预期结果
6	文件资源管理视图	访问对应文件夹下的所有文件	以资源管理器视图的方式进行页面导航	达到预期结果
7	文档在线预览	预览各种格式的文档	跳转到在线文档预览页面	达到预期结果
8	图片预览播放	文件夹下多图片分组	转入图片缩放分组播放页面	达到预期结果
9	音频播放	打开各种音频文件	能够正常在线播放音频文件	达到预期结果
10	视频播放	打开各种视频文件	能够正常在线播放视频文件	达到预期结果
11	在线解压缩	打开多层次压缩包	跳转到压缩包内文件资源浏览页面	达到预期结果

测试功能效果图，如下所示。



图 5-4 文件批量上传前--文档列表效果图

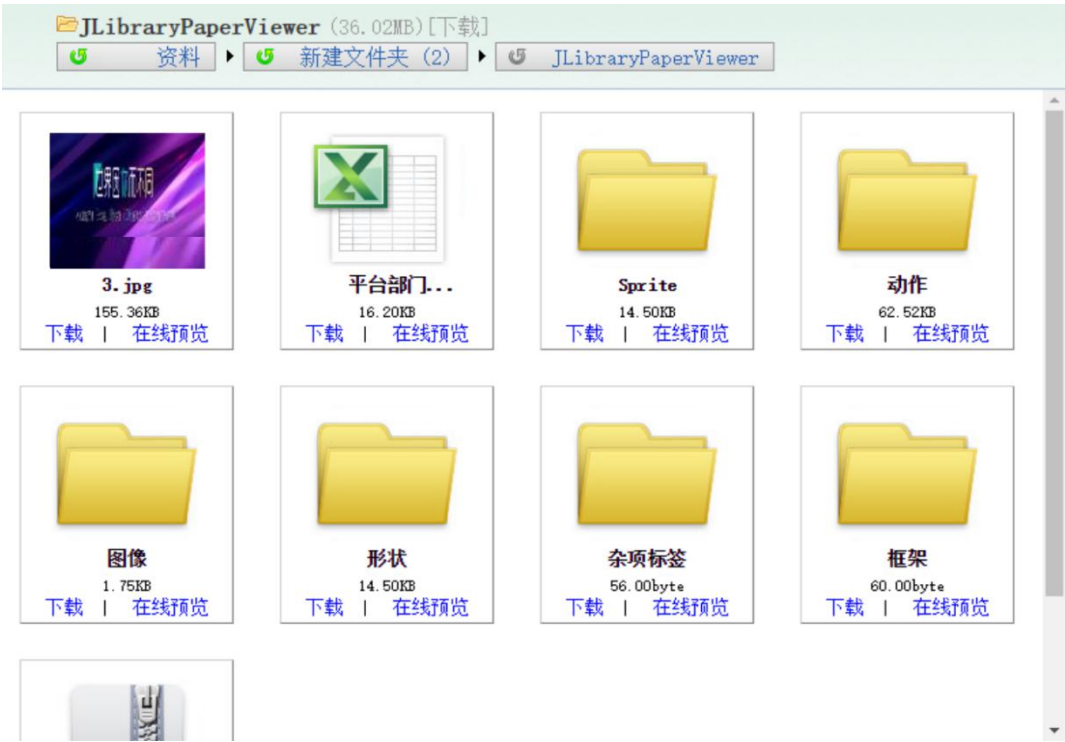


图 5-5 资源管理器--多级目录导航条菜单图

平台部门职责及目标指标.xlsx 文件长度：16.20KB 下载

年度绩效考核表（体系/部门、管理者）				
平台部 员工姓名： 岗位名称： 平台部经理 直接上级： 填表日期： 年 月 日				
计划表			考评表	
半年度目标（O）	关键指标&关键成果(KR至多5项)	权重	完成情况	上级评分
CAP+平台	1、规划完成率>=85% 2、发版延期率<15% 3、代码开发单元测试验收一次通过率>=85% 4、迭代任务完成率>=90% 5、CAP+平台示例应用>=2个	80%		
3.0老产品维护	1、客户BUG超期率：<20% 2、每周BUG超期数：<5个	20%		
总平均得分		总评等级		
评语				
示为年度/半年度，不可随意改动；关键成果可根据实际情况进行变更，最终支撑目标达成； 评分标准：每项指标评分为100分制（A+:X≥100分,A: 100>X≥95分,B+: 95>X≥90分,B: 90>X≥80分,C: 80>X≥70分,D: X<70分） 最终得分为各个KR的权重平均；				

200% 2 / 2页

图 5-6 Excel 文档转换效果图



图 5-7 图片播放效果图

5.1.4 性能测试

利用 load Runner 压力测试来对该私有网盘进行性能跟踪测试，首先要计划 load Runner 实施过程，其次设计测试用例（测试负载+用例信息），再次录制测试脚本，通过虚拟用户脚本合并到测试场景。最后执行测试反馈给出分析测试结果。具体测试用例如图 5-8 所示。

测试序号		SCS_PTUC_01	脚本名称	SCS_PTS_01
测试目的		验证大概规模并发点击服务操作时，系统性能反应情况		
测试数据及条件		10G各种格式类型文件		
脚 本 描 述	简述	采用Load Runner自带录制脚本		
	录制步骤	1、 portal 用户登录自服务门户 2、 点击服务查询按钮		
	事务	查询服务信息		
	同步点	该普通 portal 用户权限下所有服务		
测试场景描述		场景一 100 个用户并发，循环 200 次 场景二 200 个用户并发，循环 80 次 场景三 500 个用户并发，循环 150 次 场景四 800 个用户并发，循环 300 次 场景五 1000 个用户并发，循环 800 次		
测试场景结果数据		场景一 平均响应时间 1.03s 场景二 平均响应时间 0.87s 场景三 平均响应时间 0.81s 场景四 平均响应时间 1.22s 场景五 平均响应时间 1.51s		
测试结果验证		显示操作文件页面的时间不超过 3 秒， 即 TTLB<=3S		
测试环境恢复		重新启动所有服务器 重启 30 台测试客户机		

图 5-8 测试用例

由上述性能测试结果可得出，系统的性能稳定性大大提高，运行结果和预期结果相吻合。

5.2 测试分析

通过测试基本达到了系统产品质量检测的效果，具体结论如下：

1) 功能测试结论: 系统完成所有功能开发, 功能测试中所有功能均能正常运行, 实现预期功能目标。

2) 可靠性测试结论: 系统需具备容错验证, 操作界面友好, 并发访问控制得以实现, 基本满足预期的可靠性要求。

3) 系统中不足之处: 在系统设计中未能实现网盘用户角色权限的管理功能。程序功能模块组件的复用度有待更进一步提高

第六章 总结与展望

6.1 全文总结

本课题基于企业私有网盘系统业务应用需求为开发使用场景。研究主要完成了以下工作内容。

(1) 针对课题所涉及的专业知识和技术进行研究,分析了项目的可行性,对课题当前国内外的研究状况和研究成果进行了学习和总结,拟定了课题研究的方法与研究内容。

(2) 对课题研究 with 系统开发所需要的相关技术进行了较为全面的学习。包括面向对象的思想与开发技术、基于面向对象的 UML 建模方法、JavaEE 程序开发技术等内容,为后续的软件设计与开发打下基础。

(3) 以选定的相关业务人员开展需求调研,进行需求分析。完成了课题研究中系统参与者、软件功能、安全性、可靠性、扩展性等方面的需求分析,为软件的设计与开发提供可靠依据。

(4) 使用面向对象的系统设计与编程方法完成了系统的设计与开发。以 OOD 的思想和方法进行了系统的架构设计、功能设计、类设计、对象设计等概要和详细设计的内容。并用 JavaEE 开发技术,以 JAVA 这一面向对象的编程语言为主, Javascript 脚本语言、HTML 语言完成了系统的开发。

(5) 以系统需求分析和设计为基础,对完成开发的软件系统进行了功能、性能等方面的测试任务。对测试中出现的不足之处进行了一定的改进,形成了系统测试结论,完成系统开发的主要任务。

通过课题的研究,我对计算机技术的应用提升了认识,对计算机软件的设计与开发相关的知识掌握有了很大程度的提高,拓宽了我的专业知识面。对于计算机软件设计与开发的过程有了新的更全面的认识,也对当前的软件开发技术的应用有了更好的掌握,加深了技术应用的深度。软件系统的开发,从课题基础研究,技术研究,系统需求获取与分析,系统设计,系统开发,系统测试以及部署应用等全过程是一个科学的过程,需要以系统的知识体系学习为基础,全身心投入时间和精力进行课题的分析、研究。这一过程对我个人的专业水平的提升和综合能力的提高起到了很大的促进作用。使我对计算机科学技术认识从基础的应用层面提升到了科学分析、设计与应用的层面,能够将所学的专业知识体系和应用研究方法 with 具体的实践工作结合起来,解决工作中、学习研究中的实际问题。对我在今后的工作中科学的运用专业知识和技术解决实际问题有很大的启发。

6.2 后续工作展望

作为本课题研究对象的企业私有网盘系统，目前完成的各项研究和设计与开发工作是阶段性的，故而本课题研究的系统将会有持续迭代的开发过程。

本课题研究的过程中把一些可扩展性要求考虑在内，为后期需求和功能的变化打下基础。在后期工作中，根据本课题研究的结论，系统将不断进行分析和修改，以满足用户的业务需求。

致谢

在本论文完成之际，我要感谢电子科技大学让我有幸能感受这里的学习氛围和良好的生活环境，并且给学生提供了丰富的学习科研资源。

感谢我的指导老师杨国武教授，在论文的完成过程中给予了我很大的帮助，悉心指导每一个章节，提出了很多独到的见解，帮助我形成了科学的学习方法，树立了正确的研究方向。用认真严谨的科研精神，勤奋求实的职业道德，将会让我在今后的学习和工作中受用一生。在此，特向在我论文编写整个过程中的指导老师致以崇高的敬意和由衷的感谢。

我的企业导师邹莉洁，在我论文的她在学习和研究工作中给予我无私的帮助，让我感受到了企业文化和责任感。感谢亲爱的家人和朋友们在我学习期间给予的支持和鼓励。我感谢我的同学们在学习期间给予我无私的帮助和支持。

最后，衷心的感谢在百忙之中为论文评阅和答辩而付出的各位专家、教授！再次谢谢你们。

参考文献

- [1] 姚洪宇.云计算:大数据时代的系统工程[M].北京:电子工业出版社, 2013,124-125
- [2] 李浪,朱雅莉,熊江.软件工程[M].华中科技大学出版社,2013,126-127
- [3] JohnN. Howard, Michael L. kazar and Sherri G. Meness, etc. Seale and Performanee in a Distributed File System. ACM Transactionson ComPuter Systems, 1988, 6(I): 49-66
- [4] Dave McCrory. Advanced Server Virtualization: VW ware and Microsoft Platforms in the Virtual Data Center[M]. USA: Auerbach publishers Inc, 2006, 367-369
- [5] Ian Foster, YongZhao, Ioan Raieu, shiyong Lu.computing and Grid ComPuting 360 degree compared, In:Proc.of the Grid Computing Environment workshop, 2008, 3-12
- [6] Sanjay Ghemawat, Howard Gobioff, shun-Tak Leung. The Google File System[C],.Proc. Of the 19th ACM Symp. On Operating Systems Principles. 2003, 29-43
- [7] HDFS Federation 设计动机与基本原理[OL]. <http://dongxicheng.org/tag/hdfs-federation/>, 2013
- [8] X. Yang, Z. Liu. MapReduce as a Programming Model for Association Rules Algorithm on hadoop[C]. Proceedings of 3rd International Conference on information Science and Interaction Sciences(ICIS), June, 2010: 99-102
- [9] N. Li, L. Zeng, Q. He, et al. Parallel Implementation of Apriori Algorithm Based on MapReduce[C]. Proceeding of 13th ACIS International Conference on Software Engineering, Artificial Interlligence, Networking and Parallel & Distributed Computing(SNPD), Aug, 2012: 236-241
- [10] Braglia M, Frosolini M. An integrated productionroach to implement Project Management Information Systems within the Extended Enterprise[J]. international Journal of Project Management. 2014, 32(1): 18-29
- [11] 王征,刘心松,李美安.一种高效的基于可复制资源的分布式负载均衡策略[J].电子学报,2006, 34(8):1452-1455
- [12] 余思,桂小林,黄汝维,庄威.一种提高云存储中小文件存储效率的方案[J].西安交通大学学报,2011,45(06):59-63
- [13] Wang Boyang, Li Baochun, Li Hui. Oruta: Privacy_preserving public auditing for shared data in the cloud[C]//Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on.IEEE Transactions on Services Computing, 2013, 6(2);227-238

- [14] 林闯,苏文博,孟坤,等.云计算安全:架构、机制与模型评价[J].计算机学报, 2013, 36(09): 1765-1784
- [15] 冯朝胜,秦志光,袁丁.云数据安全存储技术[J]. 计算机学报,2015, 38(1):150-163
- [16] 李经纬,贾春福,刘哲理,等.可搜索加密技术研究综述[J].软件学报, 2015,26(1):109-128
- [17] 耿纪昭.云存储中数据完整性验证机制的研究与实现[D].电子科技大学. 2013
- [18] HongYong Yu, deshuai Wang. Mass log data processing and mining based on Hadoop and cloud computing. Computer Science & Education(ICCSE), 2012, 7TH International Conference, 2012, 197-202
- [19] 宋雨,易璐,王凤霞.基于云存储的重复数据删除架构的研究与设计.计算机系统的应用, 2013, 22(01):208-210
- [20] 马小龙, 刘兰娟.基于在线机制设计的私有云资源分配研究. 计算机应用研究, 2015,(02):539-542
- [21] 黄猛,罗桦,李红兵.基于高效档案馆私有云存储平台的构建.办公自动化, 2015,(01)54-59
- [22] Lisanti Y, Luhukay D, Mariani V. The design of knowledge management system model for SME(Small and Medium Enterprise) (Phase 2 - The pilot implementation in IT SMEs)[C]//international Conference on Information and Communication Technology. 2014, 9(11):211-216
- [23] M. M. Alabbadi. Cloud computer for education and learning[C]. Proceedings of 14th International Conference on Education and learning as a service (ElaaS) interactive Collaborative Learning(ICL), Piestany, Sept, 2011: 589-594
- [24] 季向远,杨涛,冯绍敏,李巧红.支持协同工作的私有云企业网盘.机械设计与制造工程, 2015,44(1):71-73
- [25] Chen Lanxiang, Zhou Shuming, Huang Xinyi, et al. Data dynamics for remote data possession checking in cloud storage[J]. Computer and Electrical Engineering, 2013, 39(7):2413-2424
- [26] Atenies G, Kamara S, Katz J. Proofs of storage from homomorphic identification protocols[C]. Proceeding of the 15th International Conference on the Theory and Application of Cryptology and Information Security,2009, Tokyo, Japan. pp. 319-333
- [27] Atenies G, Burns R, Curtmola R, et al. Provable data possession at untrusted stores[C]. Proceeding of the 14th ACM Conference on Computer and Communication Security, 2007, Alexandria, USA. pp. 598-609
- [28] 李凤华,苏铨,史国振,等.王文控制模型研究进展与发展趋势[J].电子学报, 2012, 40(4):805-813
- [29] 罗军舟,金嘉晖,宋爱波,等.云计算安全存储技术[J] 通信学报, 2011, 32(7): 3-21

- [30] 王玉林. 多节点容错存储系统的数据与缓存组织研究[D]. 电子科技大学 2010
- [31] 王于丁, 杨家海, 徐聪, 等. 云计算访问控制技术研究综述[J]. 软件学报, 2015, 26(5):1129-1150